



**COMSATS University Islamabad,  
Sahiwal Campus, Pakistan**

# **MAESTRO (Mental Assessment & Emotional Support for Therapeutic Resilience Outcomes)**

A project presented to  
**COMSATS University Islamabad**

In partial fulfillment  
of the requirement for the degree of

**Bachelor of Science in Computer Science (2022-2026)**

***By***

**Komal**

**CIIT/SP22-BCS-054/SWL**

**Muhammad Ali Asad**

**CIIT/SP22-BCS-064/SWL**

**Muhammad Nubaid Asif**

**CIIT/SP22-BCS-084/SWL**

# DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts.

If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

**Komal**

**Muhammad Ali Asad**

---

---

**Muhammad Nubaid Asif**

---

# CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) "MAESTRO" was developed by **Komal** (CIIT/SP22-BCS-054/SWL), **Muhammad Ali Asad** (CIIT/SP22-BCS-064/SWL), and **Muhammad Nubaid Asif** (CIIT/SP22-BCS-084/SWL) under the supervision of **Mr. Ahmad Shaf** and co-supervisor **Dr. Fakhar Mustafa** and that in (their/his/her) opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

---

## **Supervisor**

**Mr. Ahmad Shaf**

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

---

## **External Examiner**

**Dr. Muhammad SanaUllah**

Associate Professor

Air University Multan Campus

---

## **Head of Department**

**Dr. Zafar Iqbal**

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

# Executive Summary

The escalating global mental health crisis constitutes a significant challenge to public health systems, characterized by a widening gap between the prevalence of mental disorders and the availability of professional care. In Pakistan and globally, barriers such as social stigma, high costs, and a scarcity of qualified practitioners prevent millions from seeking timely intervention. To address this critical "treatment gap," there is an urgent need for accessible, scalable, and scientifically grounded digital health interventions.

MAESTRO is a comprehensive, intelligent mobile application designed to democratize access to mental wellness tools. Developed using the **Flutter** framework with **GetX** architecture, it offers a seamless cross-platform experience backed by a high-performance **FastAPI** backend. MAESTRO distinguishes itself through a multimodal AI engine. It integrates **Retrieval Augmented Generation (RAG)** with **Voice Analysis** capabilities. By leveraging **OpenAI Whisper** for transcription and **Librosa** for acoustic feature extraction (pitch, energy), the system's Detection Chat analyzes both text and voice inputs against verified clinical research to provide context-aware risk assessments for ten specific conditions.

Beyond active conversation, MAESTRO introduces novel methods for engagement and monitoring. To overcome the subjectivity of self-reporting, the system implements **Digital Phenotyping**, a background service that passively tracks behavioral markers such as physical activity (steps), social isolation (GPS mobility), and sleep hygiene to objectively calculate a "Risk Badge." Simultaneously, to combat user attrition, the application features the "**Mind Garden**." This gamified ecosystem visualizes the user's mental state as a virtual plant that "withers" during inactivity and "blooms" when users engage in coping mechanisms like breathing exercises or journaling.

The platform serves as a complete self-management ecosystem, incorporating a robust testing suite of **10 Clinical Assessments** (e.g., PHQ-9, GAD-7) and **4 Personality Frameworks**. Committed to user privacy, MAESTRO utilizes secure authentication and encrypted local storage to safeguard sensitive health and sensor data. By combining clinical-grade screening, passive behavioral monitoring, and gamified self-care, MAESTRO aims to function as an effective early-intervention platform, empowering users to take charge of their mental journey.

# Acknowledgement

First and foremost, all praise and gratitude are due to Almighty Allah, the Most Gracious and the Most Merciful. It is only through His blessings and the strength He bestowed upon us that we were able to complete this complex undertaking. We are humbled by the knowledge granted to us, which served as the guiding light throughout the development of MAESTRO.

We would like to express our deepest appreciation to our supervisor, Mr. Ahmad Shaf, and our co-supervisor, Dr. Fakhar Mustafa. Their profound technical insight, continuous encouragement, and constructive criticism were invaluable in shaping the direction of this project. Their mentorship helped us navigate the challenges of integrating advanced AI technologies into a mobile platform and ensured we remained focused on our objectives.

Our sincere thanks also go to our faculty members for providing us with a strong theoretical foundation in Computer Science. We are also grateful to our families for their unwavering patience and support during the long hours of coding and documentation. Their belief in our potential gave us the motivation to persevere through difficulties.

Finally, we acknowledge the contributions of our colleagues and volunteer testers. Their rigorous testing and honest feedback regarding the application's usability were instrumental in refining the user experience and ensuring the stability of the final product.

---

**Komal**  
(CIIT/SP22-BCS-054/SWL)

---

**Muhammad Ali Asad**  
(CIIT/SP22-BCS-064/SWL)

---

**Muhammad Nubaid Asif**  
(CIIT/SP22-BCS-084/SWL)

# Abbreviations

| Abbreviation | Description                        |
|--------------|------------------------------------|
| AI           | Artificial Intelligence            |
| API          | Application Programming Interface  |
| BaaS         | Backend-as-a-Service               |
| CBT          | Cognitive Behavioral Therapy       |
| GDPR         | General Data Protection Regulation |
| JSON         | JavaScript Object Notation         |
| LLM          | Large Language Model               |
| ML           | Machine Learning                   |
| MVVM         | Model-View-ViewModel               |
| NLP          | Natural Language Processing        |
| PC           | Personal Computer                  |
| RAG          | Retrieval Augmented Generation     |
| REST         | Representational State Transfer    |
| SDD          | Software Design Document           |
| SDLC         | Software Development Life Cycle    |
| SRS          | Software Requirement Specification |
| UI           | User Interface                     |
| UX           | User Experience                    |

# Table of Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>1</b>  |
| 1.1      | Brief . . . . .                                                  | 1         |
| <b>2</b> | <b>Problem Statement</b>                                         | <b>2</b>  |
| <b>3</b> | <b>Relevance to Course Modules</b>                               | <b>2</b>  |
| <b>4</b> | <b>Project Background</b>                                        | <b>3</b>  |
| <b>5</b> | <b>Objectives of the Project</b>                                 | <b>3</b>  |
| 5.1      | Development of RAG-Based AI Systems . . . . .                    | 3         |
| 5.2      | Implementation of Passive Behavioral Monitoring . . . . .        | 4         |
| 5.3      | Integration of Gamified Self-Care Tools . . . . .                | 4         |
| 5.4      | Integration of Comprehensive Assessments . . . . .               | 4         |
| 5.5      | Deployment of a Secure Hybrid Architecture . . . . .             | 4         |
| <b>6</b> | <b>Scope of the Project</b>                                      | <b>5</b>  |
| 6.1      | Frontend Mobile Application . . . . .                            | 5         |
| 6.2      | Backend and AI Architecture . . . . .                            | 5         |
| 6.3      | Data Management and Integrations . . . . .                       | 5         |
| 6.4      | Out of Scope . . . . .                                           | 5         |
| <b>7</b> | <b>Literature Review</b>                                         | <b>7</b>  |
| 7.1      | Technological Background in Mental Health Applications . . . . . | 7         |
| 7.1.1    | Evolution of Conversational Agents . . . . .                     | 7         |
| 7.1.2    | Retrieval Augmented Generation (RAG) . . . . .                   | 7         |
| 7.2      | Review of Existing Mobile Mental-Health Apps . . . . .           | 8         |
| 7.3      | Comparison with MAESTRO Capabilities . . . . .                   | 8         |
| <b>8</b> | <b>Analysis from Literature Review</b>                           | <b>8</b>  |
| 8.1      | Identified Gaps . . . . .                                        | 9         |
| 8.2      | How MAESTRO Addresses These Gaps . . . . .                       | 9         |
| 8.3      | Contribution of Literature to Project Design . . . . .           | 10        |
| <b>9</b> | <b>Methodology and Software Lifecycle</b>                        | <b>12</b> |
| 9.1      | Agile Iterative Development . . . . .                            | 12        |
| 9.2      | System Architecture Considerations . . . . .                     | 13        |
| 9.2.1    | Frontend Architecture (MVVM with GetX) . . . . .                 | 14        |
| 9.2.2    | Backend and AI Layer (RAG Pipeline) . . . . .                    | 14        |
| 9.2.3    | Data Persistence Layer . . . . .                                 | 15        |

|           |                                                                |           |
|-----------|----------------------------------------------------------------|-----------|
| 9.3       | Rationale behind Selected Methodology . . . . .                | 16        |
| <b>10</b> | <b>Problem Definition</b>                                      | <b>16</b> |
| 10.1      | Problem Statement . . . . .                                    | 16        |
| 10.2      | Deliverables and Development Requirements . . . . .            | 17        |
| 10.3      | Current System . . . . .                                       | 17        |
| 10.4      | System Limitations and Constraints . . . . .                   | 18        |
| <b>11</b> | <b>Requirement Analysis</b>                                    | <b>18</b> |
| 11.1      | Requirement Identifying Technique: Use Case Modeling . . . . . | 18        |
| 11.1.1    | System Actors . . . . .                                        | 19        |
| 11.1.2    | Functional Use Case Summaries . . . . .                        | 19        |
| 11.2      | Use Cases Diagram . . . . .                                    | 20        |
| 11.3      | Detailed Use Case Specifications . . . . .                     | 22        |
| 11.4      | Functional Requirements . . . . .                              | 38        |
| 11.4.1    | User Authentication and Onboarding . . . . .                   | 38        |
| 11.4.2    | AI-Driven Detection Chat Modules . . . . .                     | 39        |
| 11.4.3    | Self-Care, Gamification and Assessment Modules . . . . .       | 40        |
| 11.4.4    | Passive Monitoring Requirements . . . . .                      | 41        |
| 11.5      | Non-Functional Requirements . . . . .                          | 42        |
| 11.5.1    | Usability Requirements . . . . .                               | 42        |
| 11.5.2    | Performance Requirements . . . . .                             | 42        |
| 11.5.3    | Security Requirements . . . . .                                | 43        |
| 11.5.4    | Maintainability Requirements . . . . .                         | 43        |
| 11.5.5    | Emergency Handling . . . . .                                   | 44        |
| <b>12</b> | <b>Design and Architecture</b>                                 | <b>46</b> |
| 12.1      | System Architecture . . . . .                                  | 46        |
| 12.1.1    | Frontend Layer (Flutter & GetX) . . . . .                      | 46        |
| 12.1.2    | Backend Layer (FastAPI & RAG Engine) . . . . .                 | 46        |
| 12.1.3    | Architecture Diagram . . . . .                                 | 47        |
| 12.2      | Data Representation . . . . .                                  | 47        |
| 12.2.1    | Data Dictionary . . . . .                                      | 47        |
| 12.3      | Process Flow . . . . .                                         | 48        |
| 12.3.1    | Process Flow Diagrams . . . . .                                | 48        |
| 12.4      | Design Models . . . . .                                        | 52        |
| 12.4.1    | Sequence Diagrams . . . . .                                    | 52        |
| 12.4.2    | Class Diagram . . . . .                                        | 54        |
| 12.5      | Data Flow Considerations . . . . .                             | 54        |



|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>13 Implementation</b>                                          | <b>57</b> |
| 13.1 Key Algorithms and Logic . . . . .                           | 57        |
| 13.1.1 Multimodal Detection Algorithm (Audio + Text) . . . . .    | 57        |
| 13.1.2 Mind Garden Gamification Algorithm . . . . .               | 57        |
| 13.1.3 Digital Phenotyping Risk Algorithm . . . . .               | 58        |
| 13.1.4 Personality Test Scoring Algorithm . . . . .               | 58        |
| 13.1.5 Journal Entry Handling Algorithm . . . . .                 | 58        |
| 13.1.6 Mood Tracking Algorithm . . . . .                          | 58        |
| 13.1.7 Authentication and Session Algorithm . . . . .             | 59        |
| 13.2 External APIs . . . . .                                      | 59        |
| 13.3 User Interface . . . . .                                     | 60        |
| 13.4 User Interface Implementation . . . . .                      | 61        |
| <b>Traceability Matrix</b>                                        | <b>67</b> |
| <b>14 Testing and Evaluation</b>                                  | <b>69</b> |
| 14.1 Manual Testing . . . . .                                     | 69        |
| 14.1.1 System Testing . . . . .                                   | 69        |
| 14.1.2 Unit Testing . . . . .                                     | 70        |
| 14.1.3 Functional Testing . . . . .                               | 75        |
| 14.1.4 Integration Testing . . . . .                              | 77        |
| 14.2 Automated Testing . . . . .                                  | 80        |
| 14.3 Emergency Handling Testing . . . . .                         | 81        |
| <b>15 Conclusion and Future Work</b>                              | <b>84</b> |
| 15.1 Conclusion . . . . .                                         | 84        |
| 15.2 Future Work . . . . .                                        | 84        |
| 15.2.1 Reintroduction of Geospatial Therapist Discovery . . . . . | 84        |
| 15.2.2 Dialect and Regional Language Support . . . . .            | 85        |
| 15.2.3 Clinical Validation Studies . . . . .                      | 85        |
| <b>References</b>                                                 | <b>86</b> |

## List of Figures

|       |                                                                              |    |
|-------|------------------------------------------------------------------------------|----|
| 9.1   | Agile Development Lifecycle adopted for MAESTRO . . . . .                    | 13 |
| 9.2   | MVVM Architecture Pattern using Flutter and GetX . . . . .                   | 14 |
| 9.3   | RAG Pipeline Architecture: Retrieval, Augmentation, and Generation Flow . .  | 15 |
| 11.1  | Use Case Diagram: User and Admin Interactions with MAESTRO . . . . .         | 21 |
| 12.1  | Hybrid Architecture: Flutter Frontend with Firebase and FastAPI AI Backend . | 47 |
| 12.2  | Process Flow Part 1: Authentication and Navigation . . . . .                 | 49 |
| 12.3  | Process Flow Part 2: Core User Features . . . . .                            | 50 |
| 12.4  | Process Flow Part 3: Admin Panel Management . . . . .                        | 51 |
| 12.5  | Sequence Diagram: RAG-Based Detection Chat Logic . . . . .                   | 52 |
| 12.6  | Sequence Diagram: Saving a Journal Entry . . . . .                           | 53 |
| 12.7  | Sequence Diagram: Mood Tracking and Analytics Update . . . . .               | 54 |
| 12.8  | Class Diagram: Data Models and ViewModels Structure . . . . .                | 54 |
| 13.1  | Login Screen: Secure User Entry . . . . .                                    | 61 |
| 13.2  | Sign-up Screen: New Account Creation . . . . .                               | 61 |
| 13.3  | Dashboard: Main Hub . . . . .                                                | 62 |
| 13.4  | AI Chat: Risk Assessment . . . . .                                           | 62 |
| 13.5  | Clinical Tests: Screening . . . . .                                          | 63 |
| 13.6  | Personality Tests: Behavioral Insights . . . . .                             | 63 |
| 13.7  | Mood Analytics: Emotional Trends . . . . .                                   | 64 |
| 13.8  | Coping Cards: Self-Help . . . . .                                            | 64 |
| 13.9  | Breathing Menu: Selection . . . . .                                          | 65 |
| 13.10 | Active Session: Guided View . . . . .                                        | 65 |
| 13.11 | Profile: Settings & History . . . . .                                        | 66 |
| 13.12 | Journal: Rich Text Entry . . . . .                                           | 66 |

## List of Tables

|                                                                      |    |
|----------------------------------------------------------------------|----|
| 11.1 Use Case 1: User Signup . . . . .                               | 23 |
| 11.2 Use Case 2: User Login . . . . .                                | 24 |
| 11.3 Use Case 3: Detection Chat (Mental Health Assessment) . . . . . | 25 |
| 11.4 Use Case 4: Detection Test . . . . .                            | 26 |
| 11.5 Use Case 5: Journal . . . . .                                   | 27 |
| 11.6 Use Case 6: Mood Tracker . . . . .                              | 28 |
| 11.7 Use Case 7: Personality Tests . . . . .                         | 29 |
| 11.8 Use Case 8: Manage User Accounts . . . . .                      | 30 |
| 11.9 Use Case 9: Moderate Content . . . . .                          | 31 |
| 11.10 Use Case 10: Perform Breathing Exercises . . . . .             | 32 |
| 11.11 Use Case 11: View Coping Cards . . . . .                       | 33 |
| 11.12 Use Case 12: Update Tests . . . . .                            | 34 |
| 11.13 Use Case 13: Generate Reports . . . . .                        | 35 |
| 11.14 Use Case 14: View Mental Health Education . . . . .            | 36 |
| 11.15 Use Case 15: Manage Profile and Settings . . . . .             | 37 |
| 11.16 Use Case 16: View Test History . . . . .                       | 38 |
| 11.17 Functional Requirements for User Authentication . . . . .      | 38 |
| 11.18 Functional Requirements for Detection Chat . . . . .           | 39 |
| 11.19 Functional Requirements for Self-Care and Testing . . . . .    | 40 |
| 11.20 Functional Requirements for Digital Phenotyping . . . . .      | 41 |
| 11.21 Usability Requirements . . . . .                               | 42 |
| 11.22 Performance Requirements . . . . .                             | 42 |
| 11.23 Security Requirements . . . . .                                | 43 |
| 11.24 Maintainability Requirements . . . . .                         | 43 |
| 11.25 Emergency Handling Requirements . . . . .                      | 44 |
| 13.1 Details of APIs used in the project . . . . .                   | 59 |
| 13.2 Software Requirements Traceability Matrix . . . . .             | 67 |
| 14.1 Unit Tests for Authentication Logic . . . . .                   | 70 |
| 14.2 Unit Tests for Mind Garden Service . . . . .                    | 71 |
| 14.3 Unit Tests for Chat Module . . . . .                            | 71 |
| 14.4 Unit Tests for Journaling Logic . . . . .                       | 72 |
| 14.5 Unit Tests for Mood Tracker Logic . . . . .                     | 72 |
| 14.6 Unit Tests for Profile and Settings . . . . .                   | 73 |
| 14.7 Unit Tests for Clinical Assessment Logic . . . . .              | 73 |
| 14.8 Unit Tests for Personality Assessment Logic . . . . .           | 74 |
| 14.9 Unit Tests for History Module . . . . .                         | 74 |
| 14.10 Unit Tests for Self-Help Tools . . . . .                       | 75 |

|       |                                                          |    |
|-------|----------------------------------------------------------|----|
| 14.11 | Functional Tests for Authentication . . . . .            | 76 |
| 14.12 | Functional Tests for AI Chat Modules . . . . .           | 76 |
| 14.13 | Functional Tests for Assessment Modules . . . . .        | 77 |
| 14.14 | Functional Tests for Self-Care Features . . . . .        | 77 |
| 14.15 | Integration Tests: Flutter Client and Firebase . . . . . | 78 |
| 14.16 | Integration Tests: Client and AI Backend . . . . .       | 79 |
| 14.17 | Integration Tests: RAG Pipeline and Vector DB . . . . .  | 80 |
| 14.18 | Automated Testing Tools and Scope . . . . .              | 80 |
| 14.19 | Risk Detection and Trigger Logic Tests . . . . .         | 81 |
| 14.20 | Emergency Response and Notification Tests . . . . .      | 82 |

# **Chapter 1**

## **Introduction**

# 1 Introduction

The MAESTRO project focuses on designing and developing an intelligent mental-health mobile application that integrates multiple evidence-based well-being tools into a single cohesive platform. As global awareness around mental health continues to rise, individuals increasingly rely on digital technologies for early detection of emotional distress, ongoing emotional support, and self-monitoring of psychological patterns [1]. Traditional mental-health applications often provide fragmented features such as standalone chatbots, isolated mood trackers, or static assessments but rarely combine them into one unified system capable of delivering continuous, personalized, and context-aware support.

MAESTRO addresses this gap by offering a comprehensive ecosystem that blends advanced artificial intelligence, structured psychological evaluations, journaling, and therapist-locating capabilities. The application aims to empower users not only to understand their emotional states but also to receive timely interventions and reflective insights that promote healthier habits and improved self-awareness. The project leverages state-of-the-art technologies, specifically a **Retrieval Augmented Generation (RAG)** pipeline using **Qdrant** and **Sentence Transformers**, a high-performance **FastAPI** backend, and a cross-platform **Flutter** interface [2]. Through this integration of technological innovation and psychological principles, MAESTRO aspires to serve as an accessible tool for early mental-health risk identification and everyday well-being management.

## 1.1 Brief

MAESTRO is an intelligent and user-centric mobile application designed to enhance mental wellness, emotional stability, and overall self-awareness through a cohesive collection of digital well-being tools. The application integrates multiple evidence-based components into a single platform, including a suite of **4 Personality Assessments** (MBTI, Enneagram, DISC, TIPI) to help users understand their behavioral tendencies, and **10 Clinical Mental Health Tests** (e.g., PHQ-9 for Depression, GAD-7 for Anxiety, PCL-5 for PTSD) for structured screening [3].

A key strength of MAESTRO lies in its AI-driven chatbot module. The **Detection Chat** utilizes a sophisticated RAG architecture to retrieve relevant clinical data from verified research papers, allowing it to calculate real-time probability scores for ten specific mental health conditions (including OCD, Bipolar Disorder, and Panic Disorder). Additionally, the system incorporates comprehensive self-care modules, including animated **Breathing Exercises** and a digital **Coping Card** library.

From a technical perspective, MAESTRO is built using the **Flutter** framework with **GetX** for state management, enabling seamless performance on Android and iOS. The backend is powered by **FastAPI** (Python), handling complex AI inference tasks and vector database operations, while **Firebase** services manage authentication and real-time data synchronization

[4]. Together, these technologies create a unified ecosystem that empowers users to monitor mental health, reflect on personal experiences, and seek assistance when needed.

## 2 Problem Statement

Although mental-health applications have become increasingly common, most existing solutions remain limited in scope and do not provide a unified, all-in-one system for comprehensive mental-wellness support [5]. A major gap is the absence of context-aware, AI-based early detection mechanisms. Standard chatbots often rely on static scripts or generic Large Language Models (LLMs) that may hallucinate information. They rarely utilize **Retrieval Augmented Generation (RAG)** to ground their responses in verified clinical literature, limiting their ability to identify potential symptoms of depression, anxiety, or bipolar tendencies accurately.

Furthermore, many applications restrict themselves to simple conversation and do not offer continuous wellness support through intelligent agents designed to guide users through specific crises. Another significant weakness is the lack of structured and scientifically validated psychological assessment tools. Users often must rely on separate platforms to take standardized tests like the **PHQ-9** or **MBTI**, resulting in fragmented self-evaluation experiences. Similarly, critical therapeutic tools such as journaling, mood tracking with analytics, and guided breathing exercises are often missing or not integrated with other mental-health modules.

Users experiencing emotional distress are left without an accessible method to locate verified therapists within their geographical area. MAESTRO addresses these combined gaps by integrating RAG-based AI detection, daily conversational wellness support, a library of 13+ standardized psychological tests, journaling with mood analytics, and a geolocation-based therapist search system into a single, cohesive platform.

## 3 Relevance to Course Modules

This project draws upon a wide range of core Computer Science modules studied throughout the degree program, demonstrating how theoretical concepts translate into practical, real-world software solutions. The development of MAESTRO is deeply connected with **Software Engineering** principles, particularly in the application of SDLC methodologies such as Agile and Iterative development [6], which guided the sprint cycles for features like the Coping Cards and Mood Tracker.

**Artificial Intelligence and Machine Learning** play a central role, as the project utilizes a custom RAG pipeline. This involves vector embeddings (**Sentence Transformers**), similarity search algorithms within the **Qdrant** vector database, and probability tracking logic implemented in Python (scikit-learn) to interpret user input for the Detection Chat [7].

**Mobile Application Development** forms another essential foundation, specifically through

the use of **Flutter** and **Dart**. The implementation of the MVVM architecture, custom widgets (e.g., animated breathing circles), and state management (GetX) demonstrates advanced mobile engineering skills. The project also relies heavily on **Database Systems** and **Cloud Computing**, utilizing a hybrid approach with **Firebase Firestore** for user data and **Qdrant** for high-dimensional vector storage.

Additionally, concepts from **Human-Computer Interaction (HCI)** influence the design of MAESTRO's interface, ensuring that features like the "Mood Intensity Slider" and "Staggered Grid Layouts" remain intuitive and accessible. **Information Security** is another crucial area, as the project requires secure authentication flows via Firebase and encrypted local storage (Get Storage) to protect sensitive mental-health records [8].

## 4 Project Background

Mental health is a growing global issue, with millions lacking access to timely help due to stigma, cost, or a shortage of local professionals [9]. MAESTRO is designed to mitigate these barriers by providing AI-assisted early detection grounded in clinical research. Digital interventions have shown positive outcomes in emotional regulation and anxiety reduction [10]. These findings motivated the design of MAESTRO as a hybrid mental-health companion that combines the immediacy of AI support with the reliability of standardized clinical tools.

## 5 Objectives of the Project

The MAESTRO project is driven by a set of clearly defined technical and functional objectives designed to address the gaps in current digital mental health solutions. These objectives guide the development process, ensuring the final product is both clinically relevant and technically robust.

### 5.1 Development of RAG-Based AI Systems

The primary objective is to engineer a sophisticated mobile application that transcends standard chatbots by implementing a **Retrieval Augmented Generation (RAG)** pipeline. By utilizing **FastAPI** to host the logic and **Qdrant** as a vector database, the system aims to provide context-aware, hallucination-resistant responses. This involves integrating **Sentence Transformers** to match user queries with verified clinical research, ensuring that the "Detection Chat" provides safe and scientifically grounded feedback regarding the user's mental state.



## 5.2 Implementation of Passive Behavioral Monitoring

A major technical objective is to implement **Digital Phenotyping** to provide objective mental health insights. The project aims to develop background services that utilize mobile sensors to detect behavioral anomalies associated with depression and anxiety. This includes:

- **Activity Tracking:** Using the pedometer to flag sedentary behavior (daily steps < 1000).
- **Isolation Detection:** Using GPS geofencing to determine if the user has remained within a 100-meter radius of their home for extended periods.
- **Sleep Pattern Analysis:** Monitoring app usage timestamps to detect insomnia-related activity between 12:00 AM and 5:00 AM.

## 5.3 Integration of Gamified Self-Care Tools

To support daily emotional regulation and ensure long-term user engagement, the project seeks to develop the **Mind Garden**, a reactive gamification system. The objective is to create a logic engine where the state of a virtual plant (Seed, Sapling, Bloom, Withered) directly reflects the user's recent activity. This involves implementing state management logic where **Breathing Exercises** revive (water) the plant and **Journal Entries** contribute to its growth stages, thereby incentivizing consistent use of the app's coping mechanisms.

## 5.4 Integration of Comprehensive Assessments

A key goal is to consolidate fragmented psychological testing tools into a single platform. The project aims to implement a rigorous suite of **10 Clinical Mental Health Tests** (including PHQ-9, GAD-7, and PCL-5) alongside **4 Personality Assessments** (MBTI, Enneagram, DISC, TIPI). This objective involves translating standardized scoring algorithms into code to provide users with immediate, quantitative insights into their severity levels and behavioral traits without needing external apps.

## 5.5 Deployment of a Secure Hybrid Architecture

Finally, the project aims to establish a resilient and scalable infrastructure by adopting a hybrid architectural approach. This involves leveraging **Firebase** for real-time data synchronization, secure authentication, and cloud storage, while simultaneously integrating Python-based microservices for heavy AI inference. A critical aspect of this objective is ensuring strict data privacy through local encryption (GetStorage) and secure HTTPS communication channels, guaranteeing that sensitive mental health data remains protected throughout the user experience.

## 6 Scope of the Project

The scope of the MAESTRO project is defined by specific technical deliverables and functional boundaries, covering the frontend interface, the backend AI infrastructure, and the secure management of user data.

### 6.1 Frontend Mobile Application

The project encompasses the end-to-end development of a responsive mobile interface using the **Flutter** framework. This includes implementing a robust **Model-View-ViewModel (MVVM)** architecture with **GetX** state management, ensuring the application functions seamlessly on both Android and iOS devices. The scope covers the creation of all UI screens, including the dashboard, chat interfaces, and assessment forms, while supporting dynamic Light and Dark visual themes to enhance user accessibility.

### 6.2 Backend and AI Architecture

On the backend, the project involves building a dedicated **FastAPI** microservice to host the Artificial Intelligence engine. This includes the implementation of a **Retrieval Augmented Generation (RAG)** pipeline that utilizes **Sentence Transformers** for vector embedding and **Qdrant** for semantic search. The scope covers the specific logic required to detect and score 10 distinct mental health conditions based on user input, as well as the document processing logic needed to ingest clinical research papers into the vector database.

### 6.3 Data Management and Integrations

Data management is a critical deliverable, requiring the integration of **Firebase Firestore** for the secure cloud storage of user journals, test histories, and mood logs, alongside **Get Storage** for encrypted offline local caching. Furthermore, the scope includes the implementation of **Firebase Cloud Messaging** to handle scheduled notifications for daily reminders and wellness prompts. The system is designed to handle synchronization between local and cloud states to ensure data consistency.

### 6.4 Out of Scope

It is crucial to define the limitations of the system to manage expectations. The project scope explicitly excludes the provision of official medical diagnoses; the application is designed strictly as a screening and support tool, not a replacement for clinical diagnostic instruments. Additionally, while the system provides informational resources and helpline numbers, it does not offer active emergency crisis intervention services or direct integration with emergency response dispatch systems.

## **Chapter 2**

### **Literature Review**

## 7 Literature Review

The domain of digital mental health intervention has witnessed a paradigm shift in the last decade, transitioning from simple informational repositories to interactive, AI-driven support systems. This evolution addresses the critical global “treatment gap,” where the demand for psychological support far exceeds the supply of human practitioners [1]. In developing regions like Pakistan, this gap is further exacerbated by social stigma and a severe shortage of qualified professionals, leaving millions without access to timely care [9].

Existing solutions, most notably market leaders such as Yana, Sintelly, and Dr. Mind, have successfully introduced chatbot-based support mechanisms to the general public. However, a critical analysis reveals that while these platforms excel in providing basic conversational interaction, they often operate with varying degrees of clinical efficacy. They frequently lack robust, structured detection modules capable of identifying specific mental health conditions—such as distinct markers for anxiety versus depression—and rarely offer integrated systems for connecting users with professional help. MAESTRO distinguishes itself within this domain by synthesizing risk detection via **Retrieval Augmented Generation (RAG)**, wellness-focused conversation, reflective journaling, and standardized personality testing into a single, cohesive platform. This design philosophy prioritizes user privacy while ensuring that all self-help content is grounded in evidence-based psychological principles.

### 7.1 Technological Background in Mental Health Applications

The integration of Artificial Intelligence into mental health systems has gained substantial popularity, driven by three primary factors that address traditional barriers to care: accessibility, stigma reduction, and scalability [5].

#### 7.1.1 Evolution of Conversational Agents

Early mental health applications relied on rule-based chatbots (e.g., ELIZA), which used simple pattern matching. While safe, these systems lacked the ability to understand context or nuance [11]. The advent of Large Language Models (LLMs) represented a significant leap forward, allowing for more naturalistic conversations. However, standard LLMs suffer from “hallucination,” where the model confidently generates incorrect medical advice [7].

#### 7.1.2 Retrieval Augmented Generation (RAG)

To mitigate the risks of hallucination, modern architectures have shifted toward Retrieval Augmented Generation (RAG). This approach combines the generative capabilities of LLMs with a retrieval component that fetches relevant, verified information from a trusted knowledge base before generating a response. MAESTRO adopts this cutting-edge approach by utilizing **Sentence Transformers** to create vector embeddings of clinical research papers and storing

them in a **Qdrant** vector database. This ensures that every response generated by the Detection Chat is grounded in scientific literature, significantly increasing safety and reliability compared to standard generative models [12].

## 7.2 Review of Existing Mobile Mental-Health Apps

To better understand the market gap, it is essential to analyze specific market leaders such as Wysa, Youper, and Calm, all of which demonstrate the potential of conversational agents and wellness programs.

- **Wysa:** Widely recognized for its empathetic, AI-driven interface, Wysa uses a hybrid model of AI and human support [11]. However, its primary emphasis is on general emotional support (CBT-based active listening) rather than structured, condition-specific detection. It acts more as a listener than a screening tool.
- **Mentat AI:** Mentat AI utilizes AI effectively to drive conversations based on certain conditions of the user. But it's not designed for the detection purpose. There are no wellness techniques like breathing exercises in it. It also acts as a listener not as a detection and wellness platform.
- **Calm:** Focuses heavily on meditation and mindfulness. While effective for relaxation and stress reduction, it operates primarily as a preventative tool and lacks the diagnostic features necessary to identify specific mental health disorders like PTSD or Bipolar Disorder [10].

## 7.3 Comparison with MAESTRO Capabilities

MAESTRO differentiates itself from these single-purpose applications by offering a comprehensive feature set that spans the entire spectrum of mental wellness. Unlike competitors that focus solely on chat or meditation, MAESTRO combines AI-driven diagnosis-like pattern detection with evidence-based psychological self-tests (e.g., PHQ-9, GAD-7) to provide users with concrete insights into their mental state [3]. This quantitative approach is complemented by qualitative tools, including mood graphing and journal analytics, which allow for longitudinal tracking of emotional health. Crucially, MAESTRO bridges the gap between digital and physical care by including a searchable directory of therapists powered by geolocation services.

# 8 Analysis from Literature Review

A comparative analysis of the literature highlights significant distinctions between passive and active digital mental health interventions. Many current platforms provide passive support, offering information or therapeutic conversation only when prompted by the user, but they

fail to actively monitor or detect risk indicators during interactions. The literature suggests that transitioning to real-time, AI-driven detection is crucial, as it allows the system to identify emerging concerns as they happen and immediately guide users toward appropriate professional help [13]. Furthermore, the analysis indicates that isolated interactions are less effective than continuous monitoring; integrating features like journaling and mood tracking is essential because it enhances longitudinal insight, allowing both the user and the system to visualize mental health trends over time.

## 8.1 Identified Gaps

The review of existing systems revealed several critical gaps that limit the efficacy of current solutions:

1. **Hallucination Risk:** General AI chatbots lack clinical grounding, often providing generic or incorrect advice.
2. **Fragmentation:** Users are often forced to use separate applications for AI detection, journaling, testing, and finding a therapist.
3. **Lack of Real-Time Analysis:** Many apps rely on static scripts rather than dynamic NLP interpretation found in modern RAG pipelines.
4. **Limited Personalization:** Existing apps often provide generic advice rather than context-aware guidance based on specific conditions like OCD or Social Anxiety.
5. **Operational Blind Spots:** There is generally no unified dashboard available for administrators to moderate content and manage system data, which is essential for maintaining platform integrity.

## 8.2 How MAESTRO Addresses These Gaps

MAESTRO is explicitly architected to resolve these identified deficiencies through a unified, multi-module approach. To address the risk of AI hallucination, the application employs a **RAG pipeline** powered by a **FastAPI** backend and **Qdrant** database, ensuring all advice is retrieved from verified clinical documents. This is supplemented by structured, scientifically validated tests (e.g., **PHQ-9**, **GAD-7**) ensuring personalization and accuracy in assessment [14]. To solve the issue of fragmentation, MAESTRO provides accessible journaling and mood visualization tools within the same environment. Furthermore, it closes the loop on care by integrating a professional resource finder. Finally, the inclusion of a dedicated admin system ensures proper oversight, content moderation, and data management, addressing the operational gaps found in other systems.

### 8.3 Contribution of Literature to Project Design

The insights gathered from the literature played a foundational role in shaping the design and functionality of MAESTRO. The research guided the selection of specific psychological test frameworks (PHQ-9, GAD-7), ensuring that the assessments offered are valid and reliable [3]. It also supported the inclusion of journaling and mood logging modules, as the literature confirms these activities lead to positive therapeutic outcomes. Furthermore, the review reinforced the decision to use **RAG-based AI** as a primary interface, validating it as the safest and most effective method for early digital intervention. Lastly, the literature highlighted the critical importance of user needs regarding privacy (GDPR compliance) [8], empathy, and conversational naturalness, which became core non-functional requirements for the MAESTRO system.

## **Chapter 3**

# **Methodology and Requirement Analysis**



## 9 Methodology and Software Lifecycle

The development of MAESTRO adheres to the Agile Software Development Lifecycle (SDLC), specifically utilizing an iterative model to ensure flexibility and continuous improvement [13]. This approach is characterized by breaking down the project into manageable units, or sprints, allowing for the incremental development and evaluation of core modules. The process began with foundational elements such as authentication and local storage implementation, progressed to complex functionalities like the Retrieval Augmented Generation (RAG) pipeline and vector database integration, and concluded with auxiliary features such as standard psychological tests and therapist search capabilities. Furthermore, the implementation strictly follows the Object-Oriented Methodology (OOM) and the Model-View-ViewModel (MVVM) architectural pattern, ensuring that the codebase remains modular, reusable, and maintainable throughout the project's lifecycle [15].

### 9.1 Agile Iterative Development

The adoption of the Agile iterative model (Figure 9.1) provides several strategic advantages essential for the successful deployment of a complex health application. A primary benefit is the capability for rapid prototyping, which allowed the development team to visualize and test the specific AI probability scoring logic early in the process, thereby reducing the gap between design concepts and functional reality [6]. This model also facilitated the strict prioritization of high-risk modules; for instance, the Python-based FastAPI backend and Qdrant vector integration were developed and tested in earlier iterations to identify potential latency bottlenecks before they impacted the mobile frontend.

### Agile (Scrum) Development Lifecycle

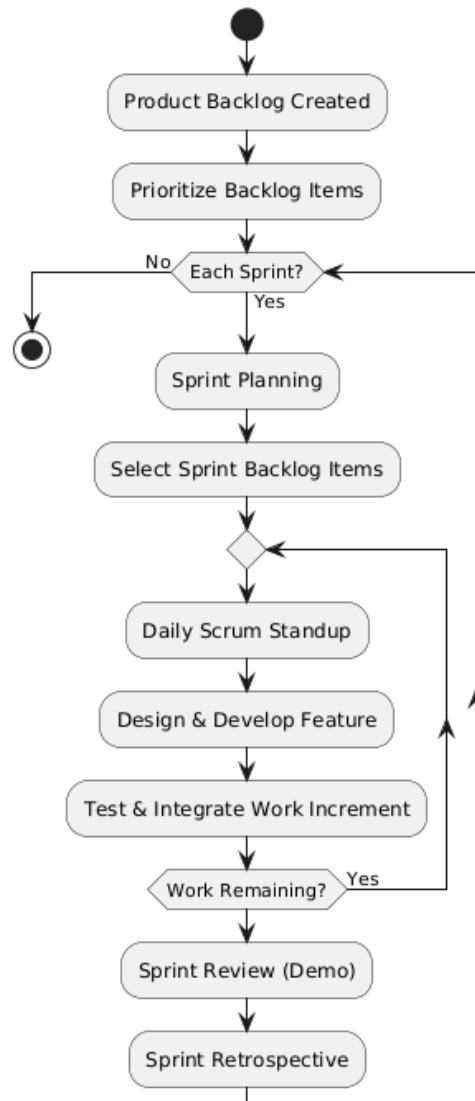


Figure 9.1: Agile Development Lifecycle adopted for MAESTRO

Additionally, the methodology supports continuous integration, ensuring that new code commits—such as the addition of new clinical test modules—are automatically tested and merged, preserving system stability. Crucially, Agile relies on stakeholder feedback loops, enabling the team to incorporate suggestions from supervisors regarding the specific phrasing of "Wellness Chat" responses during development.

## 9.2 System Architecture Considerations

The architectural framework of MAESTRO is designed to be robust, scalable, and platform-agnostic, utilizing a hybrid composition of cloud-native and microservice technologies.

### 9.2.1 Frontend Architecture (MVVM with GetX)

The frontend is constructed using the **Flutter** cross-platform framework, enabling the deployment of a single codebase to both Android and iOS devices [2]. To ensure efficient state management and logic separation, the application utilizes the **GetX** ecosystem.

This implements the **Model-View-ViewModel (MVVM)** pattern, where the *View* (UI Widgets) is decoupled from the *ViewModel* (GetX Controllers). As illustrated in Figure 9.2, the *ViewModel* mediates between the *View* and the *Model*, allowing for reactive state updates without rebuilding the entire widget tree.

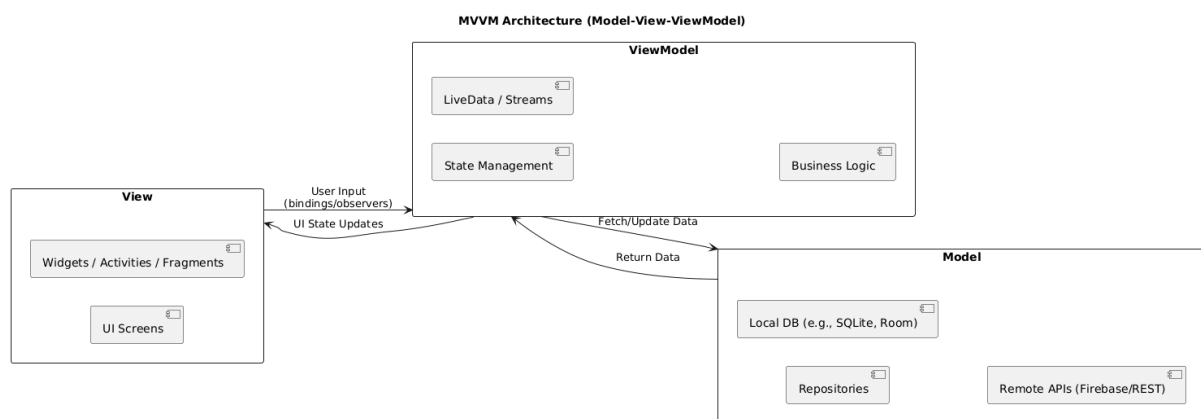


Figure 9.2: MVVM Architecture Pattern using Flutter and GetX

### 9.2.2 Backend and AI Layer (RAG Pipeline)

Unlike traditional apps that rely solely on BaaS, MAESTRO integrates a dedicated high-performance backend built with **FastAPI** (Python). This service hosts the core intelligence of the system, including the **Retrieval Augmented Generation (RAG)** engine.

The architecture employs **Qdrant** as a vector database to store high-dimensional embeddings generated by **Sentence Transformers** (all-MiniLM-L6-v2). This allows the system to perform semantic searches across ingested clinical research papers to ground the chatbot's responses in verified medical data.

Figure 9.3 details this three-step process:

1. **Retrieval:** The user's query is vectorized and matched against the clinical database.
2. **Augmentation:** The retrieved context is combined with the original prompt.
3. **Generation:** The LLM (llama-3.3-70b-versatile) generates a clinically accurate response based on the augmented prompt.

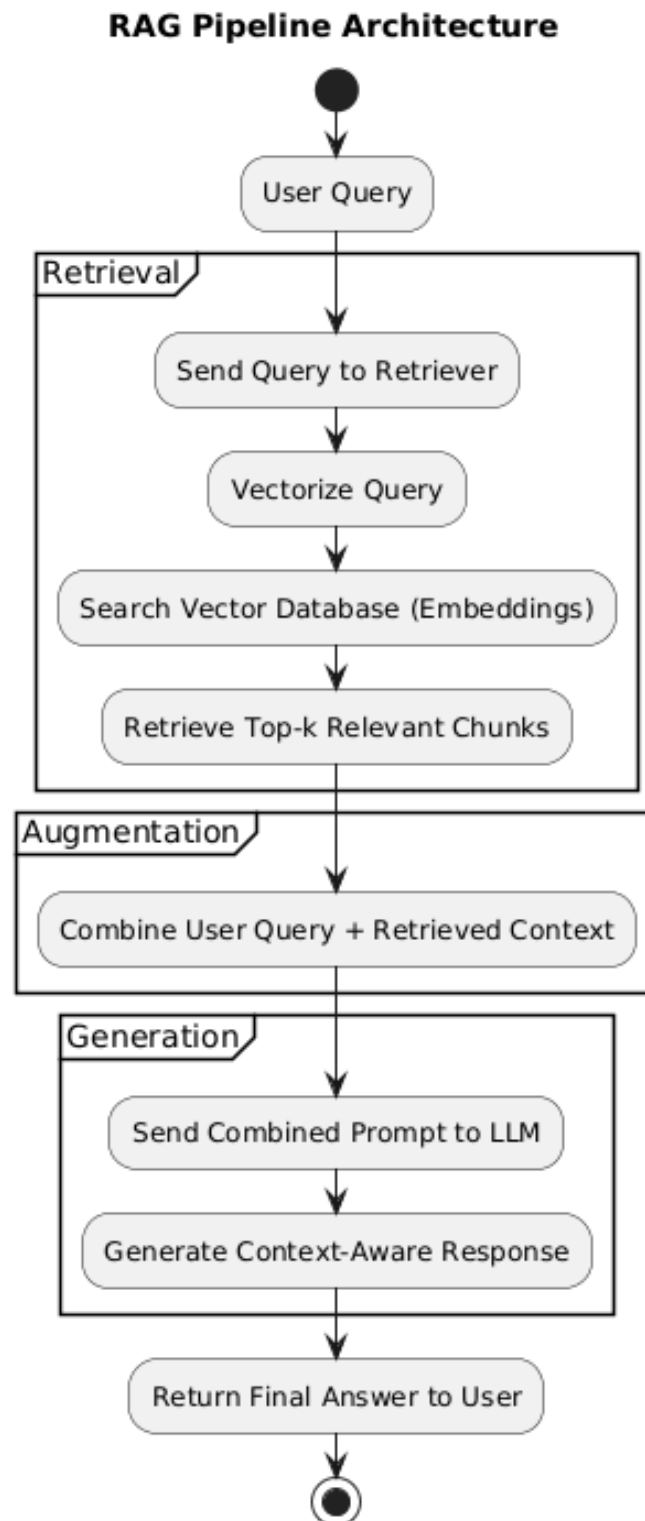


Figure 9.3: RAG Pipeline Architecture: Retrieval, Augmentation, and Generation Flow

### 9.2.3 Data Persistence Layer

The system employs a dual-database strategy to optimize for both real-time synchronization and offline availability. **Firebase Firestore** acts as the primary document store for user-centric data,

including profiles, journal entries, and test history, ensuring real-time synchronization across devices [4]. Simultaneously, **Get Storage** is utilized for encrypted local caching on the device, ensuring that users can access critical features like Coping Cards even without an active internet connection.

### **9.3 Rationale behind Selected Methodology**

The decision to combine Agile methodologies with this specific hybrid architecture is driven by the rigorous requirements of developing a mental health application. First, the use of a RAG pipeline over a standard LLM was chosen to mitigate the risk of "hallucination," ensuring that all advice provided by the Detection Chat is cited from indexed clinical PDFs [7]. Second, the selection of FastAPI allows for the seamless integration of Python's rich machine learning ecosystem (scikit-learn, PyTorch) which is required for the probability tracking of the 10 monitored mental health conditions. Finally, the iterative nature of Agile significantly simplified the integration of these disparate services (Flutter, Firebase, and Hugging Face Spaces), allowing developers to resolve cross-origin resource sharing (CORS) and latency issues in smaller, manageable batches rather than attempting a high-risk "big bang" integration at the end of the project.

## **10 Problem Definition**

This chapter delineates the specific challenges within the current mental healthcare landscape that MAESTRO intends to resolve. It provides a data-driven analysis of the treatment gap in Pakistan, defines the specific technical deliverables required to address these issues, and outlines the operational constraints within which the system must function.

### **10.1 Problem Statement**

The state of mental healthcare in Pakistan presents a critical public health crisis, with current estimates suggesting that mental health issues affect over 50 million people, representing roughly one in four individuals. Despite this staggering prevalence, approximately 90% of those in need lack access to adequate care due to a severe "treatment gap" [9]. This disparity is driven by a confluence of factors, including deep-rooted societal stigma, prohibitive financial costs, and a chronic shortage of qualified professionals. Recent data indicates that with only 0.4% of the national health budget allocated to mental health, there are fewer than 500 qualified psychiatrists available for a population exceeding 240 million [16]. Consequently, individuals face insurmountable barriers in seeking timely and effective support. The crisis is particularly acute among adolescents, with nearly 40% reporting experiences of severe anxiety stemming from academic and social pressures, often exacerbated by a lack of school-based counseling

services [17]. MAESTRO aims to address these systemic failures by providing an accessible, private, and scalable digital solution that bridges the gap between early detection, self-help tools, and professional referrals.

## 10.2 Deliverables and Development Requirements

The project is committed to delivering a comprehensive software ecosystem designed to mitigate the identified problems through a hybrid architectural approach. The primary deliverable is a robust, cross-platform mobile application built on the **Flutter** framework (utilizing **GetX** for state management), which ensures seamless functionality on both Android and iOS devices. This frontend will house a suite of integrated features including a **Detection Chat** powered by RAG, a **Wellness Chat** for casual support, and a robust self-assessment module containing **10 Clinical Tests** (e.g., PHQ-9, GAD-7) and **4 Personality Assessments** (e.g., MBTI, Enneagram).

Supporting this frontend is a sophisticated backend infrastructure divided into two distinct services. **Firebase** is utilized for secure user authentication, real-time database synchronization (Firestore), and cloud storage for media assets. Parallel to this, a high-performance **FastAPI** (Python) microservice is required to host the AI logic. This includes the **Retrieval Augmented Generation (RAG)** pipeline, which leverages a **Qdrant** vector database and **Sentence Transformers** to index and retrieve clinical research papers for context-aware chatbot responses. These technical deliverables are complemented by a full suite of documentation, including a user manual, a merged Software Requirements Specification (SRS) and Software Design Document (SDD) report, and a comprehensive set of automated test cases to validate system functionality.

## 10.3 Current System

An analysis of the current landscape reveals that there is no integrated system currently deployed for the targeted region that effectively combines structured risk detection, wellness conversations, and professional therapist recommendations with the necessary privacy guarantees. While various solutions exist, they typically operate in isolation, creating a fragmented user experience. Some applications, such as Wysa or Woebot, focus exclusively on therapy chat services but often lack localized context for Pakistan or integrated therapist directories. Others provide static informational resources or simple mood tracking without active AI engagement. There is a distinct lack of platforms that offer active detection of mental distress via **probability scoring** for specific conditions (like OCD or PTSD) coupled with an integrated referral pathway. This leaves a significant void in the continuity of care, as users who flag as "high risk" during a self-test currently have no immediate, in-app mechanism to locate nearby professional intervention.

## 10.4 System Limitations and Constraints

Despite the robust design of MAESTRO, the system operates within certain technical and operational constraints that have shaped its engineering. Primarily, the application relies on internet connectivity for its core AI features; the **RAG pipeline** and **Vector Database** queries require a stable network connection to communicate with the hosted FastAPI backend on **HuggingFace Spaces**. To mitigate this, the system employs **Get Storage** for local caching, allowing features like Journaling, Breathing Exercises, and Coping Cards to function offline, though the chatbot remains online-only.

Additionally, hardware limitations pose a constraint, as resource-limited mobile devices in developing regions may struggle with heavy animations or local processing. The architecture addresses this by offloading heavy inference tasks to the cloud. From a linguistic perspective, the initial release utilizes NLP models optimized primarily for English, meaning support for local dialects (such as Urdu or Punjabi) in the Detection Chat may be limited in nuance. Finally, the development is bound by strict legal and ethical compliance requirements. As the app handles sensitive health data, it must implement rigorous encryption protocols and clear disclaimers that the AI analysis is a screening tool, not a medical diagnosis, to comply with digital health safety standards.

## 11 Requirement Analysis

This chapter provides a comprehensive analysis of the system requirements, detailing the functional specifications and user interactions that define the MAESTRO platform. The requirements were identified and modeled using standard software engineering techniques to ensure clarity and completeness, specifically tailored to the hybrid architecture of the system.

### 11.1 Requirement Identifying Technique: Use Case Modeling

To accurately capture the functional requirements of the MAESTRO application, the Use Case modeling technique was employed. This approach provides a visual and textual representation of user interactions with the system, effectively bridging the gap between user needs and technical implementation. The Use Case Diagram serves as a blueprint for the system's behavior, identifying the various functionalities offered by the application and explicitly defining the relationships between the primary actors and the system components. By mapping these interactions, we ensure that all critical user pathways from initial authentication to complex RAG-based AI interactions are accounted for and validated against the project objectives.

### 11.1.1 System Actors

The interaction within the MAESTRO environment is driven by two primary entities. The first and central actor is the **User**, representing the individual seeking mental health support. This actor interacts with every module of the application, utilizing the platform for self-assessment, emotional regulation, and finding professional help.

The second entity is the **System**, which is now defined as a composite agent comprising the **Flutter Frontend**, the **Firestore Backend**, and the **FastAPI AI Engine**. Unlike a passive database, the System actively manages state via GetX, executes complex inference tasks using a Retrieval Augmented Generation (RAG) pipeline, and queries a **Qdrant** vector database to retrieve clinical context. It is responsible for calculating real-time probability scores, triggering notifications, and dynamically updating content based on the user's emotional state.

### 11.1.2 Functional Use Case Summaries

The core functionalities of the system are categorized into distinct use cases, each addressing a specific aspect of mental wellness.

**User Authentication and Session Management** The entry point to the application involves a secure authentication process designed to protect sensitive user data. This functionality is twofold: first, the "Sign Up" process allows new users to create an account using email credentials or Google Sign-In integration. Second, the "Login" function grants registered users access by validating their identity against the Firebase Authentication server. The system also manages persistent sessions and secure token exchange to ensure user data remains protected across app restarts.

**RAG-Based Mental Health Detection** A pivotal function of the application is the automated detection of mental health conditions. In this scenario, the system utilizes a **Retrieval Augmented Generation (RAG)** pipeline. When a user engages in the Detection Chat, the system embeds their responses using Sentence Transformers and queries the Qdrant vector database. This allows the AI to analyze inputs against indexed clinical research papers, generating real-time probability scores for ten specific conditions, including Depression, Anxiety, PTSD, OCD, and Bipolar Disorder.

**Emotional Support and Wellness Chat** Users can engage with the system to receive immediate emotional support via the Wellness Chat. This use case involves an interactive dialogue where the AI agent provides empathetic responses and general wellness advice. Unlike the detection module, this chat focuses on de-escalation and daily coping, offering a safe space for users to vent their feelings without fear of judgment.



**Interactive Self-Regulation Tools** To assist users in managing acute stress, the system includes interactive Coping Cards and Breathing Exercises. The Coping Cards module allows users to swipe through categorized strategies for issues like Panic Attacks or Anger Management. Simultaneously, the Breathing Exercise function guides users through animated routines—such as Box Breathing (4-4-4-4) or 4-7-8 Breathing—complete with visual cues and audio feedback to promote immediate physiological relaxation.

**Comprehensive Clinical and Personality Testing** The system offers a robust suite of 14 standardized assessments. The **Clinical Testing** use case covers 10 specific screening tools (e.g., PHQ-9 for Depression, GAD-7 for Anxiety, PCL-5 for PTSD), providing users with a severity score (Minimal to Severe). Complementing this, the **Personality Testing** use case allows users to explore their behavioral traits through 4 distinct frameworks: MBTI, Enneagram, DISC, and the TIPI Big Five assessment.

**Journaling and Mood Analytics** To foster self-reflection, the application includes modules for logging journal entries and tracking moods. The journaling function allows users to record daily thoughts with rich text and image support, stored securely in the cloud. The Mood Tracker enables users to select from five animated mood categories and adjust intensity on a 1-10 scale. The system then aggregates this data to visualize emotional trends over weekly, monthly, and yearly views using interactive line charts.

## 11.2 Use Cases Diagram

The Use Case Diagram in Figure 11.1 visualizes the complete scope of user interactions. It delineates the system boundary, illustrating how the primary **User** actor interacts with the core modules like Authentication, AI Chat, and Self-Care while also highlighting the administrative functions available to the **Admin** actor.

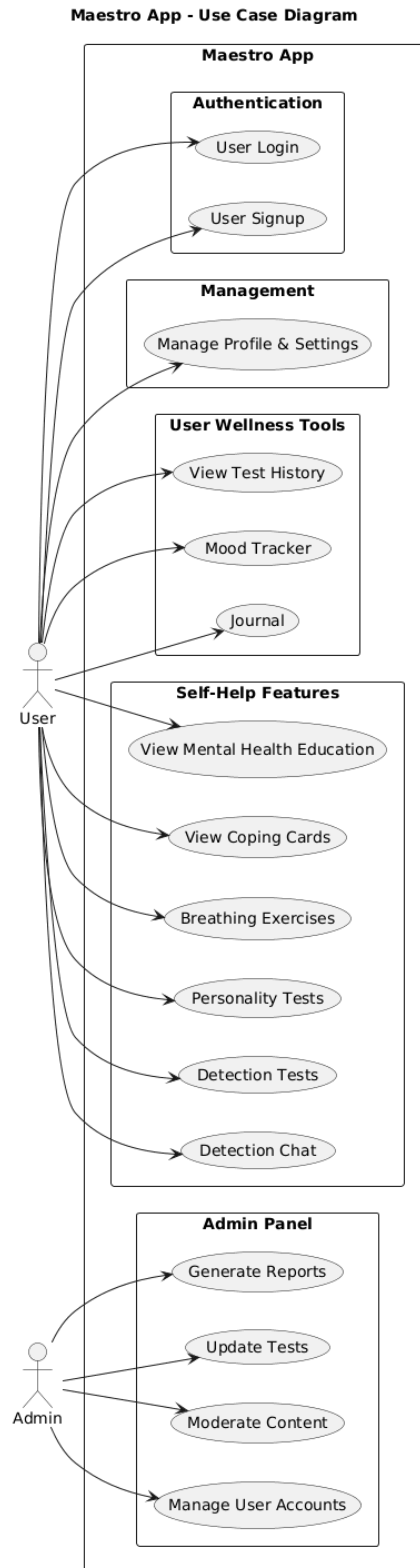


Figure 11.1: Use Case Diagram: User and Admin Interactions with MAESTRO

### **11.3 Detailed Use Case Specifications**

To provide a granular view of the system's behavior, detailed use case descriptions (UC-1 to UC-14) are documented. These specifications elaborate on the preconditions, postconditions, and step-by-step flow of events for every identified function. They cover User Signup, User Login, RAG-based Detection Chat, Wellness Chat logic, Therapist Finder, execution of the 13 Clinical and Personality Tests, and the management of Journal and Mood Tracker data. Furthermore, administrative use cases such as Managing User Accounts, Moderating Content, and Generating System Reports are included to define the backend operational workflows.

#### **Use Case 1: User Signup**

The User Signup process is the initial entry point for new users, enabling them to create a secure account within the MAESTRO ecosystem. By providing essential credentials such as an email address and password, users establish a unique identity that allows the system to securely store and retrieve their personal data, mental health records, and preferences across sessions. The detailed specifications for this use case are provided in Table 11.1.

Table 11.1: Use Case 1: User Signup

|                          |                                                                                                                                                                                                                                                                                                                                                                             |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-1                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Use Case Name</b>     | User Signup                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Actors</b>            | Primary Actor: User                                                                                                                                                                                                                                                                                                                                                         |
| <b>Description</b>       | A new user registers an account to access the MAESTRO application.                                                                                                                                                                                                                                                                                                          |
| <b>Trigger</b>           | User selects the signup option on the application.                                                                                                                                                                                                                                                                                                                          |
| <b>Preconditions</b>     | <ol style="list-style-type: none"> <li>1. User has a valid email or phone number.</li> <li>2. User has a stable internet connection.</li> </ol>                                                                                                                                                                                                                             |
| <b>Postconditions</b>    | <ol style="list-style-type: none"> <li>1. User account is successfully created and stored in Firebase.</li> <li>2. User is redirected to the login screen.</li> </ol>                                                                                                                                                                                                       |
| <b>Normal Flow</b>       | <ol style="list-style-type: none"> <li>1. User selects 'Sign Up'.</li> <li>2. System prompts for user details (name, email/phone, password).</li> <li>3. User enters details and submits the form.</li> <li>4. System validates the input.</li> <li>5. System stores user credentials securely in Firebase.</li> <li>6. System redirects user to the login page.</li> </ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"> <li>1. If email is already registered, system prompts an error.</li> <li>2. If password is short, system prompts user to create a stronger password.</li> </ol>                                                                                                                                                                          |

### Use Case 2: User Login

The User Login functionality serves as the secure gateway for returning users to access the platform's features. This process authenticates the user's identity against the stored credentials in the database, ensuring that sensitive personal health information remains private and accessible only to the authorized account holder. The detailed specifications for this use case are provided in Table 11.2.

Table 11.2: Use Case 2: User Login

|                          |                                                                                                                                                                                                                                                                                                                                   |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-2                                                                                                                                                                                                                                                                                                                              |
| <b>Use Case Name</b>     | User Login                                                                                                                                                                                                                                                                                                                        |
| <b>Actors</b>            | Primary Actor: User                                                                                                                                                                                                                                                                                                               |
| <b>Description</b>       | A registered user logs into the MAESTRO application.                                                                                                                                                                                                                                                                              |
| <b>Trigger</b>           | User selects the login option.                                                                                                                                                                                                                                                                                                    |
| <b>Preconditions</b>     | <ol style="list-style-type: none"> <li>1. User must have a registered account.</li> <li>2. Internet connection is required.</li> </ol>                                                                                                                                                                                            |
| <b>Postconditions</b>    | <ol style="list-style-type: none"> <li>1. User is successfully logged in and redirected to the dashboard.</li> </ol>                                                                                                                                                                                                              |
| <b>Normal Flow</b>       | <ol style="list-style-type: none"> <li>1. User selects 'Login'.</li> <li>2. System prompts for email/phone and password.</li> <li>3. User enters details and submits form.</li> <li>4. System verifies credentials against Firebase database.</li> <li>5. If credentials match, system grants access to the dashboard.</li> </ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"> <li>1. If incorrect password is entered, system prompts an error.</li> <li>2. If user forgets password, system provides a reset option.</li> </ol>                                                                                                                                             |
| <b>Exceptions</b>        | <ol style="list-style-type: none"> <li>1. If the system experiences downtime, it notifies the user and retries later.</li> </ol>                                                                                                                                                                                                  |

### Use Case 3: Detection Chat (Mental Health Assessment)

The Detection Chat leverages RAG technology to screen for 10 mental health conditions. It supports multimodal input, allowing users to communicate via text or voice. The system utilizes OpenAI Whisper for transcription and Librosa for acoustic analysis (pitch, energy) to refine the risk probability score. The detailed specifications are provided in Table 11.3.

Table 11.3: Use Case 3: Detection Chat (Mental Health Assessment)

|                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-3                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Use Case Name</b>     | Detection Chat (Voice-Enabled)                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Actors</b>            | Primary Actor: User                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Description</b>       | The AI chatbot screens for specific conditions (e.g., Anxiety, PTSD). It processes text input and voice recordings (converted via ffmpeg). Audio is analyzed for semantic content (Whisper) and acoustic emotional markers (Librosa) to inform the RAG pipeline.                                                                                                                                                                                                               |
| <b>Trigger</b>           | User initiates a conversation with the chatbot.                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Preconditions</b>     | <ol style="list-style-type: none"> <li>1. User is logged in.</li> <li>2. Internet connection is required.</li> <li>3. Microphone permissions granted (if using voice).</li> </ol>                                                                                                                                                                                                                                                                                              |
| <b>Postconditions</b>    | <ol style="list-style-type: none"> <li>1. System provides a probability score based on combined text/acoustic data.</li> </ol>                                                                                                                                                                                                                                                                                                                                                 |
| <b>Normal Flow</b>       | <ol style="list-style-type: none"> <li>1. User selects 'Detection Chat'.</li> <li>2. Chatbot initiates symptom exploration phase.</li> <li>3. User responds via text OR records an audio message.</li> <li>4. If Audio: Backend converts file (ffmpeg), transcribes (Whisper), and extracts emotional features (Librosa).</li> <li>5. System queries vector database using the processed input.</li> <li>6. System displays probability scores and recommendations.</li> </ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"> <li>1. If user mentions self-harm (in text or audio), system triggers emergency protocol.</li> </ol>                                                                                                                                                                                                                                                                                                                                        |
| <b>Exceptions</b>        | <ol style="list-style-type: none"> <li>1. If backend API is unreachable, user is prompted to check connection.</li> <li>2. If audio format is unsupported, system prompts user to retry.</li> </ol>                                                                                                                                                                                                                                                                            |

#### Use Case 4: Detection Test

Users can take 10 standardized clinical assessments (e.g., PHQ-9, GAD-7) to screen for specific disorders. The system calculates severity scores and provides immediate, evidence-based feedback. The detailed specifications for this use case are provided in Table 11.4.

Table 11.4: Use Case 4: Detection Test

|                          |                                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-4                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Use Case Name</b>     | Take Detection Test                                                                                                                                                                                                                                                                                                                                                  |
| <b>Actors</b>            | Primary Actor: User                                                                                                                                                                                                                                                                                                                                                  |
| <b>Description</b>       | Administers standardized screenings (PHQ-9, GAD-7, etc.) to assess symptom severity and provide clinical feedback.                                                                                                                                                                                                                                                   |
| <b>Trigger</b>           | The user selects a specific clinical test from the menu.                                                                                                                                                                                                                                                                                                             |
| <b>Preconditions</b>     | <ol style="list-style-type: none"> <li>1. The user is logged in.</li> <li>2. Internet required to sync results to cloud history.</li> </ol>                                                                                                                                                                                                                          |
| <b>Postconditions</b>    | <ol style="list-style-type: none"> <li>1. Test score (e.g., 15/27) is calculated and saved.</li> <li>2. Severity level (Mild/Moderate/Severe) is displayed.</li> </ol>                                                                                                                                                                                               |
| <b>Normal Flow</b>       | <ol style="list-style-type: none"> <li>1. User selects a test (e.g., PHQ-9 Depression).</li> <li>2. System presents sequential multiple-choice questions.</li> <li>3. User selects answers based on frequency of symptoms.</li> <li>4. System sums values using clinical scoring logic.</li> <li>5. Result screen displays severity and coping resources.</li> </ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"> <li>1. If user quits early, progress is discarded to ensure validity.</li> </ol>                                                                                                                                                                                                                                                  |

### Use Case 5: Journal

The Journaling feature provides a secure, private space for users to record daily thoughts. It supports rich text and image attachments, helping users process emotions and maintain a history of their mental state. The detailed specifications for this use case are provided in Table 11.5.

Table 11.5: Use Case 5: Journal

|                          |                                                                                                                                                                                                                                                                                                           |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-5                                                                                                                                                                                                                                                                                                      |
| <b>Use Case Name</b>     | Write Journal Entry                                                                                                                                                                                                                                                                                       |
| <b>Actors</b>            | Primary Actor: User                                                                                                                                                                                                                                                                                       |
| <b>Description</b>       | Records personal reflections with title, content, and optional image attachments.                                                                                                                                                                                                                         |
| <b>Trigger</b>           | The user taps the "Add Entry" button.                                                                                                                                                                                                                                                                     |
| <b>Preconditions</b>     | 1. The user is logged in.                                                                                                                                                                                                                                                                                 |
| <b>Postconditions</b>    | - The entry is encrypted and saved to Firestore and local cache.                                                                                                                                                                                                                                          |
| <b>Normal Flow</b>       | <ol style="list-style-type: none"> <li>1. User opens journal module and taps "+".</li> <li>2. User inputs title and body text.</li> <li>3. User optionally attaches an image or selects a mood tag.</li> <li>4. User saves the entry.</li> <li>5. Entry appears in the chronological timeline.</li> </ol> |
| <b>Alternative Flows</b> | 1. User can edit or delete existing entries.                                                                                                                                                                                                                                                              |
| <b>Exceptions</b>        | 1. If saving fails, data is cached locally for retry.                                                                                                                                                                                                                                                     |

### Use Case 6: Mood Tracker

Allows users to log daily emotions using animated icons and an intensity slider. The system aggregates this data to display weekly and monthly mood trends, helping users identify emotional patterns. The detailed specifications for this use case are provided in Table 11.6.



Table 11.6: Use Case 6: Mood Tracker

|                          |                                                                                                                                                                                    |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-6                                                                                                                                                                               |
| <b>Use Case Name</b>     | Track Mood                                                                                                                                                                         |
| <b>Actors</b>            | Primary Actor: User                                                                                                                                                                |
| <b>Description</b>       | Users log their current mood (Very Sad to Very Happy) and intensity (1-10) to track emotional trends.                                                                              |
| <b>Trigger</b>           | User selects the "Check-in" option.                                                                                                                                                |
| <b>Preconditions</b>     | 1. User is logged in.                                                                                                                                                              |
| <b>Postconditions</b>    | 1. Mood data is saved to Firebase.<br>2. Analytics charts are updated.                                                                                                             |
| <b>Normal Flow</b>       | 1. User opens Mood Tracker.<br>2. User selects a mood icon (e.g., Happy).<br>3. User adjusts intensity slider (1-10).<br>4. User adds optional notes/tags.<br>5. User saves entry. |
| <b>Alternative Flows</b> | 1. User can view historical charts (Weekly/Monthly).                                                                                                                               |
| <b>Exceptions</b>        | - If offline, entry saves locally and syncs later.                                                                                                                                 |

### Use Case 7: Personality Tests

Administers 4 distinct personality frameworks (MBTI, Enneagram, DISC, TIPI) to generate detailed behavioral profiles and self-awareness insights for the user. The detailed specifications for this use case are provided in Table 11.7.

Table 11.7: Use Case 7: Personality Tests

|                          |                                                                                                                                                                                                                                                                                                                             |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-7                                                                                                                                                                                                                                                                                                                        |
| <b>Use Case Name</b>     | Personality Tests                                                                                                                                                                                                                                                                                                           |
| <b>Actors</b>            | Primary Actor: User                                                                                                                                                                                                                                                                                                         |
| <b>Description</b>       | Users take assessments like MBTI or Enneagram to receive a detailed psychological profile and behavioral analysis.                                                                                                                                                                                                          |
| <b>Trigger</b>           | The user selects a specific test (e.g., "DISC Assessment").                                                                                                                                                                                                                                                                 |
| <b>Preconditions</b>     | <ol style="list-style-type: none"> <li>1. The user is logged in.</li> <li>2. Test content is loaded from the database.</li> </ol>                                                                                                                                                                                           |
| <b>Postconditions</b>    | <ol style="list-style-type: none"> <li>1. Personality type (e.g., "INFJ" or "Type 4") is saved.</li> <li>2. Detailed report is generated and displayed.</li> </ol>                                                                                                                                                          |
| <b>Normal Flow</b>       | <ol style="list-style-type: none"> <li>1. User selects a personality module.</li> <li>2. System presents a series of behavioral questions.</li> <li>3. User submits answers.</li> <li>4. System maps answers to specific personality traits.</li> <li>5. Result screen displays the user's type and description.</li> </ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"> <li>1. User can retake the test to update their profile.</li> </ol>                                                                                                                                                                                                                      |
| <b>Exceptions</b>        | <ol style="list-style-type: none"> <li>1. If network fails, results are cached locally.</li> </ol>                                                                                                                                                                                                                          |

### Use Case 8: Manage User Accounts

Allows administrators to oversee the user base, including viewing profiles, handling access issues, and enforcing bans for policy violations to ensure platform safety. The detailed specifications for this use case are provided in Table 11.8.

Table 11.8: Use Case 8: Manage User Accounts

|                          |                                                                                                                                                                                                                                                                                                             |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-8                                                                                                                                                                                                                                                                                                        |
| <b>Use Case Name</b>     | Manage User Accounts                                                                                                                                                                                                                                                                                        |
| <b>Actors</b>            | Primary Actor: Admin                                                                                                                                                                                                                                                                                        |
| <b>Description</b>       | Admin manages user accounts, including viewing, editing, and deactivating accounts.                                                                                                                                                                                                                         |
| <b>Trigger</b>           | Admin selects the user management option in the dashboard.                                                                                                                                                                                                                                                  |
| <b>Preconditions</b>     | 1. Admin is logged in with appropriate permissions.                                                                                                                                                                                                                                                         |
| <b>Postconditions</b>    | 1. User account status is updated in the database.                                                                                                                                                                                                                                                          |
| <b>Normal Flow</b>       | <ol style="list-style-type: none"> <li>1. Admin selects 'Manage User Accounts'.</li> <li>2. System displays a list of registered users.</li> <li>3. Admin searches for a specific user.</li> <li>4. Admin updates status (e.g., Deactivate).</li> <li>5. System confirms and applies the change.</li> </ol> |
| <b>Alternative Flows</b> | 1. If user not found, system displays "No results".                                                                                                                                                                                                                                                         |
| <b>Exceptions</b>        | 1. If database is unreachable, operation fails with error.                                                                                                                                                                                                                                                  |

### Use Case 9: Moderate Content

Enables administrators to review flagged journal entries or forum posts (if applicable) to remove inappropriate material and maintain community guidelines. The detailed specifications for this use case are provided in Table 11.9.

Table 11.9: Use Case 9: Moderate Content

|                          |                                                                                                                                                                                                                                                                                 |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-9                                                                                                                                                                                                                                                                            |
| <b>Use Case Name</b>     | Moderate Content                                                                                                                                                                                                                                                                |
| <b>Actors</b>            | Primary Actor: Admin                                                                                                                                                                                                                                                            |
| <b>Description</b>       | Admin reviews and moderates user-generated content, including reported messages or posts.                                                                                                                                                                                       |
| <b>Trigger</b>           | Admin accesses the content moderation panel.                                                                                                                                                                                                                                    |
| <b>Preconditions</b>     | 1. Content has been flagged for review.                                                                                                                                                                                                                                         |
| <b>Postconditions</b>    | 1. Content is either approved or deleted.                                                                                                                                                                                                                                       |
| <b>Normal Flow</b>       | <ol style="list-style-type: none"> <li>1. Admin opens moderation queue.</li> <li>2. System displays flagged item.</li> <li>3. Admin reviews content against guidelines.</li> <li>4. Admin selects "Remove" or "Keep".</li> <li>5. System updates content visibility.</li> </ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"> <li>1. If queue is empty, system shows "All clear".</li> </ol>                                                                                                                                                                               |
| <b>Exceptions</b>        | <ol style="list-style-type: none"> <li>1. If action fails, error message is shown.</li> </ol>                                                                                                                                                                                   |

### Use Case 10: Perform Breathing Exercises

Guides users through structured breathing patterns (e.g., Box Breathing) using visual animations and audio cues to induce immediate relaxation. The detailed specifications for this use case are provided in Table 11.10.

Table 11.10: Use Case 10: Perform Breathing Exercises

|                          |                                                                                                                                                                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-10                                                                                                                                                                                                                                  |
| <b>Use Case Name</b>     | Perform Breathing Exercises                                                                                                                                                                                                            |
| <b>Actors</b>            | Primary Actor: User                                                                                                                                                                                                                    |
| <b>Description</b>       | Users follow an animated guide for breathing techniques like 4-7-8 or Box Breathing to reduce stress.                                                                                                                                  |
| <b>Trigger</b>           | User selects "Meditation & Breathing" from the menu.                                                                                                                                                                                   |
| <b>Preconditions</b>     | 1. User is logged in.                                                                                                                                                                                                                  |
| <b>Postconditions</b>    | 1. Session duration is logged.<br>2. User reports feeling calmer (optional feedback).                                                                                                                                                  |
| <b>Normal Flow</b>       | 1. User selects a technique (e.g., "Box Breathing").<br>2. System displays instruction overlay.<br>3. User starts the exercise.<br>4. Animation guides inhalation, holding, and exhalation.<br>5. Session ends, and progress is saved. |
| <b>Alternative Flows</b> | 1. User can customize breath duration in settings.                                                                                                                                                                                     |
| <b>Exceptions</b>        | 1. If audio fails, visual cues remain active.                                                                                                                                                                                          |

### Use Case 11: View Coping Cards

Provides immediate, bite-sized strategies for managing acute distress (e.g., panic attacks) through a swipeable card interface organized by category. The detailed specifications for this use case are provided in Table 11.11.

Table 11.11: Use Case 11: View Coping Cards

|                          |                                                                                                                                                 |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-11                                                                                                                                           |
| <b>Use Case Name</b>     | View Coping Cards                                                                                                                               |
| <b>Actors</b>            | Primary Actor: User                                                                                                                             |
| <b>Description</b>       | Users browse categorized cards (e.g., "Anxiety Relief") containing quick, actionable advice.                                                    |
| <b>Trigger</b>           | User selects "Coping Cards" from the home screen.                                                                                               |
| <b>Preconditions</b>     | 1. None (Available offline via local storage).                                                                                                  |
| <b>Postconditions</b>    | 1. User learns a coping mechanism.<br>2. Card can be bookmarked for quick access.                                                               |
| <b>Normal Flow</b>       | 1. User opens module.<br>2. User selects category (e.g., "Panic Attack").<br>3. User swipes through cards.<br>4. User bookmarks a helpful card. |
| <b>Alternative Flows</b> | 1. User searches for specific keywords (e.g., "Sleep").                                                                                         |

### Use Case 12: Update Tests

Enables administrators to modify test questions, scoring logic, or add new assessment modules to keep the clinical tools up-to-date with current research. The detailed specifications for this use case are provided in Table 11.12.

Table 11.12: Use Case 12: Update Tests

|                          |                                                                                                                                                                                                                                                                                        |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-12                                                                                                                                                                                                                                                                                  |
| <b>Use Case Name</b>     | Update Tests                                                                                                                                                                                                                                                                           |
| <b>Actors</b>            | Primary Actor: Admin                                                                                                                                                                                                                                                                   |
| <b>Description</b>       | Admin updates or modifies mental health assessment tests.                                                                                                                                                                                                                              |
| <b>Trigger</b>           | Admin selects the 'Update Tests' option.                                                                                                                                                                                                                                               |
| <b>Preconditions</b>     | <ol style="list-style-type: none"> <li>1. Admin is logged in.</li> <li>2. Test data exists in the system.</li> </ol>                                                                                                                                                                   |
| <b>Postconditions</b>    | <ol style="list-style-type: none"> <li>1. Tests are updated or modified successfully.</li> <li>2. System logs changes for record-keeping.</li> </ol>                                                                                                                                   |
| <b>Normal Flow</b>       | <ol style="list-style-type: none"> <li>1. Admin selects 'Update Tests'.</li> <li>2. System displays available tests.</li> <li>3. Admin selects a test to modify.</li> <li>4. Admin edits the test details and submits.</li> <li>5. System updates the test in the database.</li> </ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"> <li>1. If no tests exist, system prompts to create new ones.</li> <li>2. If admin lacks permissions, system denies access.</li> </ol>                                                                                                               |

### Use Case 13: Generate Reports

Administrators can generate analytical reports on user engagement, test completion rates, and demographic trends to inform system improvements. The detailed specifications for this use case are provided in Table 11.13.

Table 11.13: Use Case 13: Generate Reports

|                       |                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>    | UC-13                                                                                                                                                                                                                                                                                                                                                     |
| <b>Use Case Name</b>  | Generate Reports                                                                                                                                                                                                                                                                                                                                          |
| <b>Actors</b>         | Primary Actor: Admin                                                                                                                                                                                                                                                                                                                                      |
| <b>Description</b>    | Admin generates system reports related to user activity, mental health assessments, and platform engagement.                                                                                                                                                                                                                                              |
| <b>Trigger</b>        | Admin selects the ‘Generate Reports’ option.                                                                                                                                                                                                                                                                                                              |
| <b>Preconditions</b>  | <ol style="list-style-type: none"> <li>1. Admin is logged in.</li> <li>2. System contains relevant data for reporting.</li> </ol>                                                                                                                                                                                                                         |
| <b>Postconditions</b> | <ol style="list-style-type: none"> <li>1. Reports are generated successfully and stored for future reference.</li> <li>2. System logs the report generation.</li> </ol>                                                                                                                                                                                   |
| <b>Normal Flow</b>    | <ol style="list-style-type: none"> <li>1. Admin selects ‘Generate Reports’.</li> <li>2. System prompts admin to choose report type and date range.</li> <li>3. Admin selects the required options and submits the request.</li> <li>4. System processes the request and generates the report.</li> <li>5. Admin downloads or views the report.</li> </ol> |

#### Use Case 14: View Mental Health Education

Users can access a library of educational resources detailing symptoms, causes, and treatment options for 10 specific mental health conditions, promoting mental health literacy. The detailed specifications for this use case are provided in Table 11.14.



Table 11.14: Use Case 14: View Mental Health Education

|                          |                                                                                                                                                                                                                                                                                  |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-14                                                                                                                                                                                                                                                                            |
| <b>Use Case Name</b>     | View Mental Health Education                                                                                                                                                                                                                                                     |
| <b>Actors</b>            | Primary Actor: User                                                                                                                                                                                                                                                              |
| <b>Description</b>       | Users browse detailed informational pages about conditions like Anxiety, Bipolar Disorder, and PTSD.                                                                                                                                                                             |
| <b>Trigger</b>           | User selects "Mental Conditions" from the dashboard.                                                                                                                                                                                                                             |
| <b>Preconditions</b>     | 1. None (Content is available offline).                                                                                                                                                                                                                                          |
| <b>Postconditions</b>    | 1. User views symptoms and treatment guidelines.                                                                                                                                                                                                                                 |
| <b>Normal Flow</b>       | <ol style="list-style-type: none"> <li>1. User navigates to the Education module.</li> <li>2. User selects a condition (e.g., "OCD").</li> <li>3. System displays text, symptoms, and help guides.</li> <li>4. User navigates to related resources or external links.</li> </ol> |
| <b>Alternative Flows</b> | <ol style="list-style-type: none"> <li>1. User uses the search bar to find specific topics.</li> </ol>                                                                                                                                                                           |

### Use Case 15: Manage Profile and Settings

Enables users to customize their app experience, including updating personal details, managing notification preferences (e.g., daily check-in reminders), and toggling UI themes (Dark/Light mode).The detailed specifications for this use case are provided in Table 11.15.

Table 11.15: Use Case 15: Manage Profile and Settings

|                          |                                                                                                                                                                                                                 |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>       | UC-15                                                                                                                                                                                                           |
| <b>Use Case Name</b>     | Manage Profile and Settings                                                                                                                                                                                     |
| <b>Actors</b>            | Primary Actor: User                                                                                                                                                                                             |
| <b>Description</b>       | Users update their profile information and configure app preferences like notifications and themes.                                                                                                             |
| <b>Trigger</b>           | User taps the "Settings" icon.                                                                                                                                                                                  |
| <b>Preconditions</b>     | 1. User is logged in.                                                                                                                                                                                           |
| <b>Postconditions</b>    | 1. Settings are saved locally and synced to the cloud.<br>2. App UI updates immediately (e.g., theme change).                                                                                                   |
| <b>Normal Flow</b>       | 1. User opens Settings.<br>2. User selects an option (e.g., "Notifications").<br>3. User sets a time for the "Daily Mood Reminder".<br>4. System schedules the local notification.<br>5. User confirms changes. |
| <b>Alternative Flows</b> | 1. User selects "Dark Mode" to toggle visual theme.                                                                                                                                                             |

### Use Case 16: View Test History

Allows users to track their mental health progress by reviewing past results from clinical and personality tests. This historical view helps users identify trends or improvements over time. The detailed specifications for this use case are provided in Table 11.16.

Table 11.16: Use Case 16: View Test History

|                       |                                                                                                                                                                                                                                                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case ID</b>    | UC-16                                                                                                                                                                                                                                                                                                                                 |
| <b>Use Case Name</b>  | View Test History                                                                                                                                                                                                                                                                                                                     |
| <b>Actors</b>         | Primary Actor: User                                                                                                                                                                                                                                                                                                                   |
| <b>Description</b>    | Users view a chronological list of past test scores and severity interpretations.                                                                                                                                                                                                                                                     |
| <b>Trigger</b>        | User selects "History" within the Test module.                                                                                                                                                                                                                                                                                        |
| <b>Preconditions</b>  | 1. User has completed at least one test previously.                                                                                                                                                                                                                                                                                   |
| <b>Postconditions</b> | 1. Historical data is retrieved from Firebase and displayed.                                                                                                                                                                                                                                                                          |
| <b>Normal Flow</b>    | <ol style="list-style-type: none"> <li>1. User navigates to Test History.</li> <li>2. System fetches past records (Date, Test Name, Score).</li> <li>3. User taps a specific record.</li> <li>4. System displays the detailed breakdown of that attempt.</li> <li>5. User visualizes progress via simple trend indicators.</li> </ol> |
| <b>Exceptions</b>     | 1. If no history exists, system prompts user to take their first test.                                                                                                                                                                                                                                                                |

## 11.4 Functional Requirements

The functional requirements define the specific behaviors and functions the MAESTRO system must support to meet user needs. These are detailed below, categorized by their respective modules.

### 11.4.1 User Authentication and Onboarding

This module establishes the secure entry pathway for users, managing identity verification, session persistence, and account recovery to ensure data privacy and authorized access. The details are provided in Table 11.17.

Table 11.17: Functional Requirements for User Authentication

| <b>Identifier</b> | <b>Title</b> | <b>Requirement</b>                                                                               | <b>Priority</b> |
|-------------------|--------------|--------------------------------------------------------------------------------------------------|-----------------|
| FR-01             | User Signup  | The system shall allow new users to register using email/password or Google Sign-In credentials. | High            |

|       |                   |                                                                                                           |        |
|-------|-------------------|-----------------------------------------------------------------------------------------------------------|--------|
| FR-02 | Email Validation  | The system shall verify the validity of provided email addresses before account activation.               | High   |
| FR-03 | Secure Login      | The system shall authenticate users against Firebase and maintain persistent sessions for ease of access. | High   |
| FR-04 | Password Recovery | The system shall provide a "Forgot Password" flow to reset credentials via email link.                    | Medium |
| FR-05 | User Onboarding   | The system shall present an introductory onboarding flow to explain core features to first-time users.    | Medium |

#### 11.4.2 AI-Driven Detection Chat Modules

These requirements define the core intelligence of the system, including the new voice processing pipeline specific to risk assessment.

Table 11.18: Functional Requirements for Detection Chat

| Identifier | Title                       | Requirement                                                                                                                            | Priority |
|------------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-06      | RAG Detection Logic         | The Detection Chat shall use Retrieval Augmented Generation to analyze user input (text or transcribed audio) against clinical papers. | Critical |
| FR-07      | Voice Input for Detection   | The Detection Chat interface shall allow users to record audio, which the backend converts to .wav via <b>ffmpeg</b> for analysis.     | High     |
| FR-08      | Acoustic Feature Extraction | For audio inputs, the system shall use <b>Librosa</b> to extract pitch and energy metrics to assist in detecting emotional distress.   | Medium   |
| FR-09      | Hybrid Risk Scoring         | The system shall calculate probability scores by combining semantic analysis (text) and acoustic analysis (voice) where available.     | High     |

|       |                   |                                                                                                                           |          |
|-------|-------------------|---------------------------------------------------------------------------------------------------------------------------|----------|
| FR-10 | Emergency Trigger | If keywords (text or voice transcript) related to self-harm are detected, the system must trigger the emergency protocol. | Critical |
|-------|-------------------|---------------------------------------------------------------------------------------------------------------------------|----------|

### 11.4.3 Self-Care, Gamification and Assessment Modules

This section outlines the suite of interactive tools designed for user self-regulation, monitoring, and engagement. It covers standardized testing, gamified rewards, and therapeutic exercises. The details are provided in Table 11.19.

Table 11.19: Functional Requirements for Self-Care and Testing

| Identifier | Title                  | Requirement                                                                                                                       | Priority |
|------------|------------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------|
| FR-09      | Clinical Testing       | The system shall administer 9 standardized clinical tests (e.g., PHQ-9, GAD-7) and calculate severity scores.                     | High     |
| FR-10      | Personality Profiling  | The system shall provide 4 personality assessments (MBTI, Enneagram, DISC, TIPI) and generate behavioral reports.                 | Medium   |
| FR-11      | Advanced Mood Tracking | The system shall allow users to log mood intensity (1-10) and view trends via weekly/monthly line charts.                         | High     |
| FR-12      | Digital Journaling     | Users shall be able to create rich-text journal entries with support for image attachments and mood tagging.                      | Medium   |
| FR-13      | Breathing Exercises    | The system shall provide animated guides for breathing techniques (e.g., 4-7-8) with audio cues.                                  | High     |
| FR-14      | Mind Garden Logic      | The system shall visually wither the plant if the user is inactive for 3 days and revive it upon completing a breathing exercise. | High     |

|       |                     |                                                                                                                 |     |
|-------|---------------------|-----------------------------------------------------------------------------------------------------------------|-----|
| FR-15 | Educational Content | The system must provide a library of information regarding symptoms and treatments for common mental disorders. | Low |
|-------|---------------------|-----------------------------------------------------------------------------------------------------------------|-----|

#### 11.4.4 Passive Monitoring Requirements

This section defines the requirements for the Digital Phenotyping service, which operates in the background to collect objective behavioral data. The details are provided in Table 11.20.

Table 11.20: Functional Requirements for Digital Phenotyping

| Identifier | Title               | Requirement                                                                                                             | Priority |
|------------|---------------------|-------------------------------------------------------------------------------------------------------------------------|----------|
| FR-16      | Step Tracking       | The system shall track daily steps using the device pedometer and flag sedentary behavior (< 1000 steps).               | High     |
| FR-17      | Isolation Detection | The system shall monitor GPS location to detect if the user remains within 100m of their home for extended periods.     | High     |
| FR-18      | Sleep Hygiene       | The system shall detect active app usage between 12 AM and 5 AM as an indicator of insomnia/anxiety.                    | Medium   |
| FR-19      | Risk Calculation    | The system shall aggregate sensor data to calculate a "Risk Level" (Low, Medium, High) and display it on the dashboard. | High     |

## 11.5 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system, such as usability, security, and performance. These constraints ensure that the MAESTRO application is not only functional but also reliable, secure, and responsive under real-world conditions, adhering to industry standards like IEEE 830 [18].

### 11.5.1 Usability Requirements

These requirements ensure the application is accessible and easy to navigate for a diverse user base, incorporating modern UI standards like theming and accessibility support in line with ISO 9241 guidelines [19].The details are provided in Table 11.21.

Table 11.21: Usability Requirements

| Identifier | Title         | Requirement                                                                                                                        | Priority |
|------------|---------------|------------------------------------------------------------------------------------------------------------------------------------|----------|
| USE-01     | Intuitive UI  | The system shall implement standard Material Design patterns to ensure a learning curve of less than 5 minutes for new users [20]. | High     |
| USE-02     | Accessibility | The application shall support screen readers and dynamic text sizing for visually impaired users.                                  | Medium   |
| USE-03     | Theme Support | The system shall provide both Light and Dark modes to reduce eye strain and accommodate user preferences.                          | High     |

### 11.5.2 Performance Requirements

To guarantee a smooth user experience, especially during AI interactions, the system must adhere to specific benchmarks regarding latency and resource efficiency.The details are provided in Table 11.22

Table 11.22: Performance Requirements

| Identifier | Title       | Requirement                                                                                              | Priority |
|------------|-------------|----------------------------------------------------------------------------------------------------------|----------|
| PERF-01    | UI Response | Navigation between screens (e.g., Dashboard to Chat) shall complete within 200ms to ensure fluidity [2]. | High     |

|         |                    |                                                                                                                   |      |
|---------|--------------------|-------------------------------------------------------------------------------------------------------------------|------|
| PERF-02 | RAG Latency        | The AI chatbot shall retrieve context and generate a response within 5–8 seconds under normal network conditions. | High |
| PERF-03 | Offline Capability | The system shall load cached content (Journals, Coping Cards) in under 1 second when offline.                     | High |

### 11.5.3 Security Requirements

Given the sensitive nature of mental health data, stringent security measures are mandatory. These requirements cover encryption standards for data at rest and in transit, following OWASP best practices [21]. The details are provided in Table 11.23.

Table 11.23: Security Requirements

| Identifier | Title            | Requirement                                                                                                     | Priority |
|------------|------------------|-----------------------------------------------------------------------------------------------------------------|----------|
| SEC-01     | Data Encryption  | Sensitive local data shall be encrypted using AES standards via the GetStorage secure container [8].            | High     |
| SEC-02     | Secure Transport | All communication between the app and the FastAPI backend shall be secured via HTTPS/TLS 1.2+ [22].             | High     |
| SEC-03     | Session Security | User sessions shall be managed via secure Firebase tokens with automatic expiration after prolonged inactivity. | Medium   |

### 11.5.4 Maintainability Requirements

These requirements focus on the code structure, ensuring the software remains scalable and easy to update by future developers. The details are provided in Table 11.24.

Table 11.24: Maintainability Requirements

| Identifier | Title                | Requirement                                                                                             | Priority |
|------------|----------------------|---------------------------------------------------------------------------------------------------------|----------|
| MAIN-01    | Modular Architecture | The code shall strictly follow the MVVM pattern withGetX to separate UI logic from business logic [15]. | High     |



|         |                         |                                                                                                                    |        |
|---------|-------------------------|--------------------------------------------------------------------------------------------------------------------|--------|
| MAIN-02 | Code Documen-<br>tation | All Python backend functions and Dart classes must include docstrings explaining parameters and return types [13]. | Medium |
| MAIN-03 | Scalability             | The backend shall be containerized (e.g., Docker/Hugging Face Spaces) to support horizontal scaling.               | High   |

### 11.5.5 Emergency Handling

Safety is critical in mental health applications. This section outlines the protocols for detecting high-risk inputs and providing immediate assistance. The details are provided in Table 11.25.

Table 11.25: Emergency Handling Requirements

| Identifier | Title                        | Requirement                                                                                                            | Priority |
|------------|------------------------------|------------------------------------------------------------------------------------------------------------------------|----------|
| EMH-01     | Risk Detection               | The RAG engine shall prioritize risk assessment, flagging keywords related to self-harm with 99% accuracy [12].        | Critical |
| EMH-02     | Emergency UI                 | Upon detecting danger, the system shall override the chat interface to display a prominent "Get Help" modal.           | Critical |
| EMH-03     | Resource Avail-<br>ability   | Emergency hotline numbers and coping resources must be stored locally to be available even without internet [5].       | High     |
| EMH-04     | False Positive<br>Management | The AI probability threshold for emergency triggers shall be tuned to minimize false alarms while prioritizing safety. | Medium   |

## **Chapter 4**

# **System Design and Architecture**

## 12 Design and Architecture

This chapter provides a comprehensive technical breakdown of the MAESTRO application's internal structure. It details the hybrid architectural pattern that integrates mobile frontend engineering with high-performance AI microservices, ensuring the system is scalable, secure, and responsive.

### 12.1 System Architecture

MAESTRO adopts a distributed, hybrid architecture that combines the **Model-View-ViewModel (MVVM)** pattern on the frontend with a microservices-based backend. This separation of concerns allows for independent scaling of the user interface and the resource-intensive AI processing layers.

#### 12.1.1 Frontend Layer (Flutter & GetX)

The client-side application is built using the **Flutter** framework, ensuring cross-platform consistency. The architecture utilizes the **GetX** ecosystem for state management, dependency injection, and route management.

- **View Layer:** Consists of reactive UI widgets (screens) that listen to state changes.
- **ViewModel (Controller) Layer:** GetX Controllers (e.g., 'DetectionChatController', 'MoodViewModel') handle business logic, input validation, and mediate data flow between the UI and the backend repositories.
- **Model Layer:** Dart classes defining data structures (e.g., 'TestResult', 'JournalEntry') with JSON serialization methods.

#### 12.1.2 Backend Layer (FastAPI & RAG Engine)

Unlike traditional BaaS-only apps, MAESTRO incorporates a dedicated **FastAPI** (Python) backend hosted on Hugging Face Spaces to manage complex AI tasks.

- **API Gateway:** Exposes REST endpoints (e.g., '/chat', '/predict') for the mobile app.
- **RAG Pipeline:** A specialized module that embeds user queries using **Sentence Transformers** and retrieves relevant clinical context from the vector database.
- **Vector Database:** **Qdrant** is used to store and query high-dimensional embeddings of verified mental health research papers, ensuring the chatbot's advice is grounded in science.

### 12.1.3 Architecture Diagram

The diagram 12.1 illustrates the data flow between the Flutter client, Firebase services (Auth/Firestore), and the external Python/FastAPI AI engine.

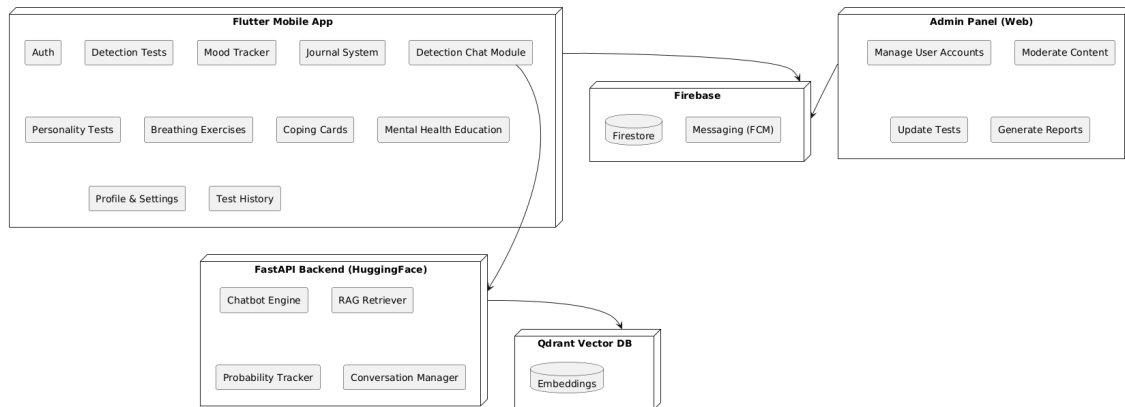


Figure 12.1: Hybrid Architecture: Flutter Frontend with Firebase and FastAPI AI Backend

## 12.2 Data Representation

### 12.2.1 Data Dictionary

The data schema is designed to support diverse content types, from structured clinical assessments to unstructured journal text and sensor logs.

- **User:** 'uid' (String), 'email' (String), 'displayName' (String), 'photoUrl' (String), 'onboardingComplete' (Bool).
- **MindGardenState:** 'waterLevel' (Int: 0-100), 'flowerStage' (Int: 1-5), 'isWithered' (Bool), 'currentStreak' (Int), 'lastActiveDate' (DateTime).
- **PhenotypingLog:** 'date' (DateTime), 'dailySteps' (Int), 'mobilityStatus' (Enum: Home/Outdoors), 'isLateNightUsage' (Bool), 'riskLevel' (Enum: Low/Med/High).
- **JournalEntry:** 'id' (String), 'title' (String), 'content' (String), 'moodTag' (String), 'timestamp' (DateTime), 'images' (List<String>).
- **MoodLog:** 'id' (String), 'moodType' (Enum: VeryHappy...VerySad), 'intensity' (Int: 1-10), 'note' (String), 'timestamp' (DateTime).
- **TestResult:** 'id' (String), 'testType' (String: PHQ-9, GAD-7), 'score' (Int), 'severity' (String), 'date' (DateTime).
- **ChatMessage:** 'id' (String), 'sender' (Enum: User/AI), 'text' (String), 'isEmergency' (Bool), 'timestamp' (DateTime).

- **VoiceAnalysisLog:** 'id' (String), 'transcription' (String), 'detectedPitch' (Float), 'energyLevel' (Float), 'tempoBPM' (Float), 'ffmpegConversionStatus' (Bool).
- **CopingCard:** 'id' (String), 'category' (String), 'title' (String), 'content' (String), 'isFavorite' (Bool).

## 12.3 Process Flow

The process flow diagram details the user's journey, specifically highlighting the decision logic within the AI Chat modules—where the system decides between a standard response, a RAG-based retrieval, or an emergency intervention trigger.

### 12.3.1 Process Flow Diagrams

The following diagrams illustrate the detailed operational workflows of the MAESTRO application, segregated into three logical parts: Authentication, Core User Features, and Administrative Management.

#### Part 1: Authentication and Navigation

Figure 12.2 illustrates the initial entry pathway for the application. It depicts the decision logic for routing new users through the onboarding and signup process, while directing returning users to the secure login screen. The flow converges at the Home Dashboard, which serves as the central navigation hub for all application features.

## Maestro App - Process Flow (Part 1: Authentication & Navigation)

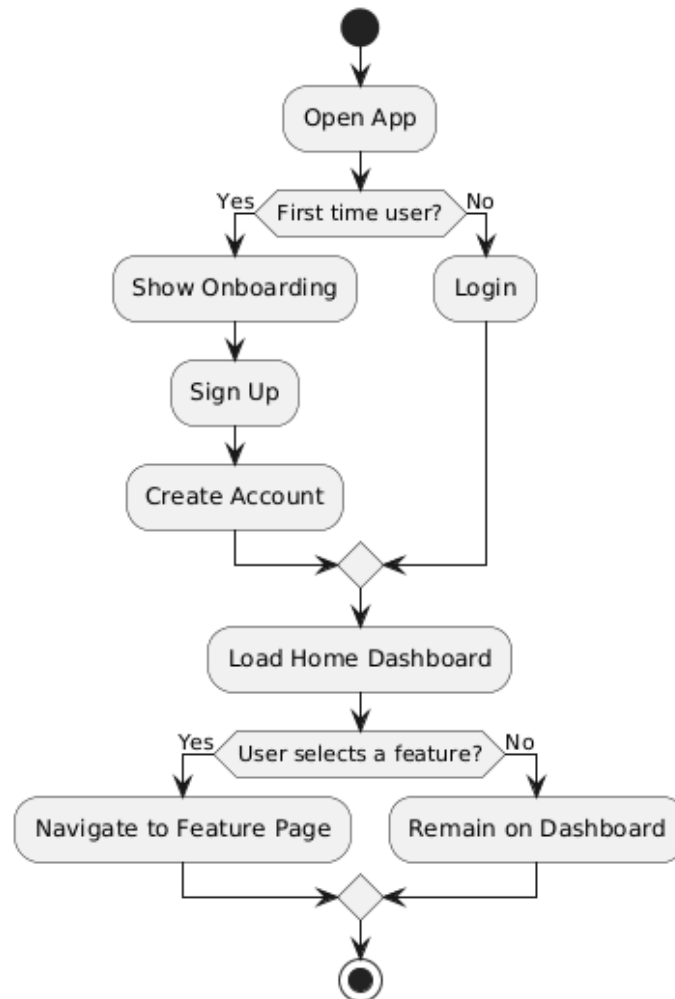


Figure 12.2: Process Flow Part 1: Authentication and Navigation

### Part 2: Core User Features

Figure 12.3 details the comprehensive operational workflow for the application's main modules. It maps the user's journey through specific features such as the RAG-based Detection Chat, standardized Clinical Tests, Mood Tracking, and Self-Care tools (Journaling, Breathing Exercises). Each branch represents a discrete user interaction cycle, from initiation to data persistence.

Maestro App - Process Flow (Part 2: Core Features)

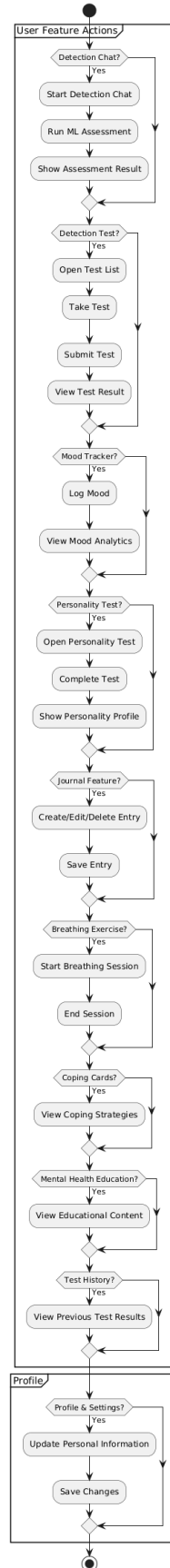


Figure 12.3: Process Flow Part 2: Core User Features

### Part 3: Admin Panel Management

Figure 12.4 outlines the administrative workflows designed to maintain system integrity. It visualizes the privileges granted to authenticated administrators, including the management of user accounts, moderation of flagged content, updating of clinical test protocols, and the generation of analytical system reports.

Maestro App - Process Flow (Part 3: Admin Panel)

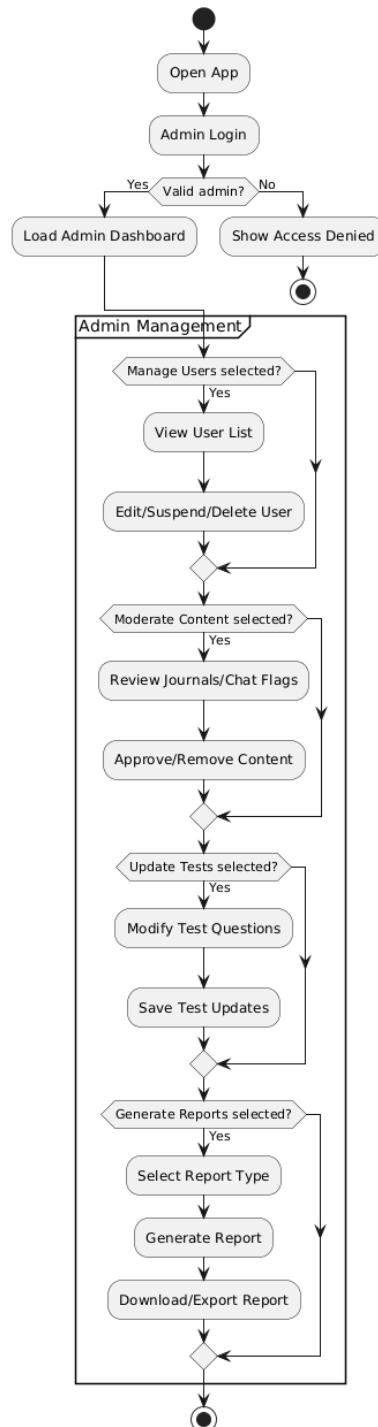


Figure 12.4: Process Flow Part 3: Admin Panel Management



## 12.4 Design Models

Standard UML artifacts define the static structure and dynamic behavior of the system components.

### 12.4.1 Sequence Diagrams

The Sequence Diagrams presented below model the dynamic interactions between the user, the frontend application (Flutter/GetX), and the various backend services (FastAPI/Firebase). They visualize the temporal order of message exchanges required to execute critical system workflows, ranging from complex AI inference to secure data persistence.

#### 1. Detection Chat Interaction

Figure 12.5 visualizes the sophisticated RAG pipeline. It traces the path of a user message from the UI → GetX Controller → API Service → FastAPI Backend. It details how the system queries the Qdrant vector database to retrieve clinical context before generating a response and updating the risk probability scores.

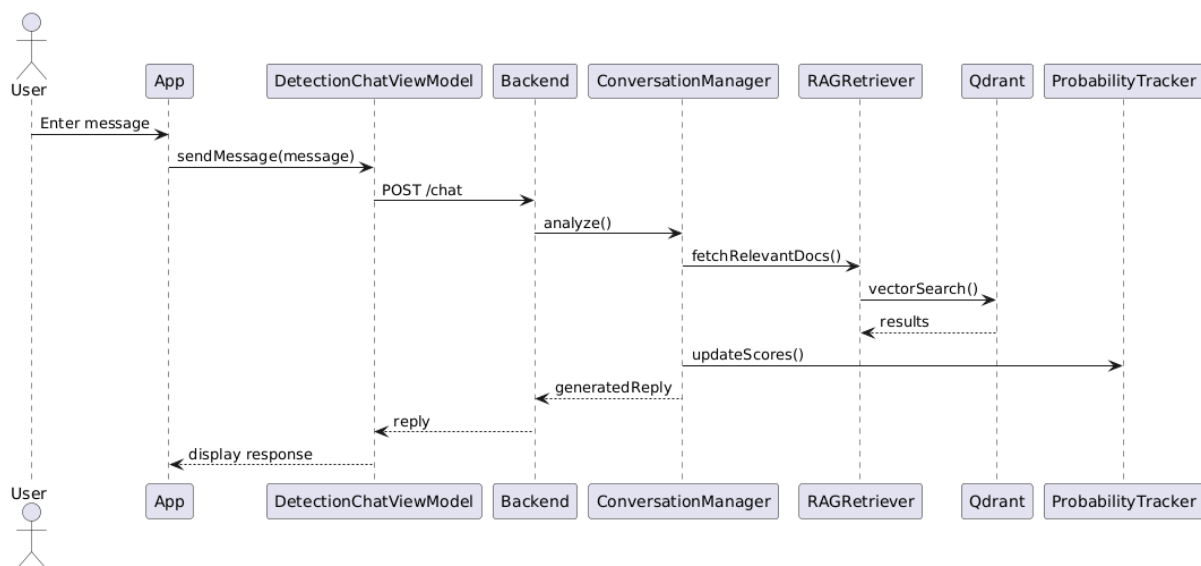


Figure 12.5: Sequence Diagram: RAG-Based Detection Chat Logic

#### 2. Journal Entry Creation

Figure 12.6 illustrates the data persistence flow for the journaling feature. It maps the sequence from the user input → ViewModel validation → encryption → and final storage in the Firebase Firestore database, concluding with a list refresh on the UI.

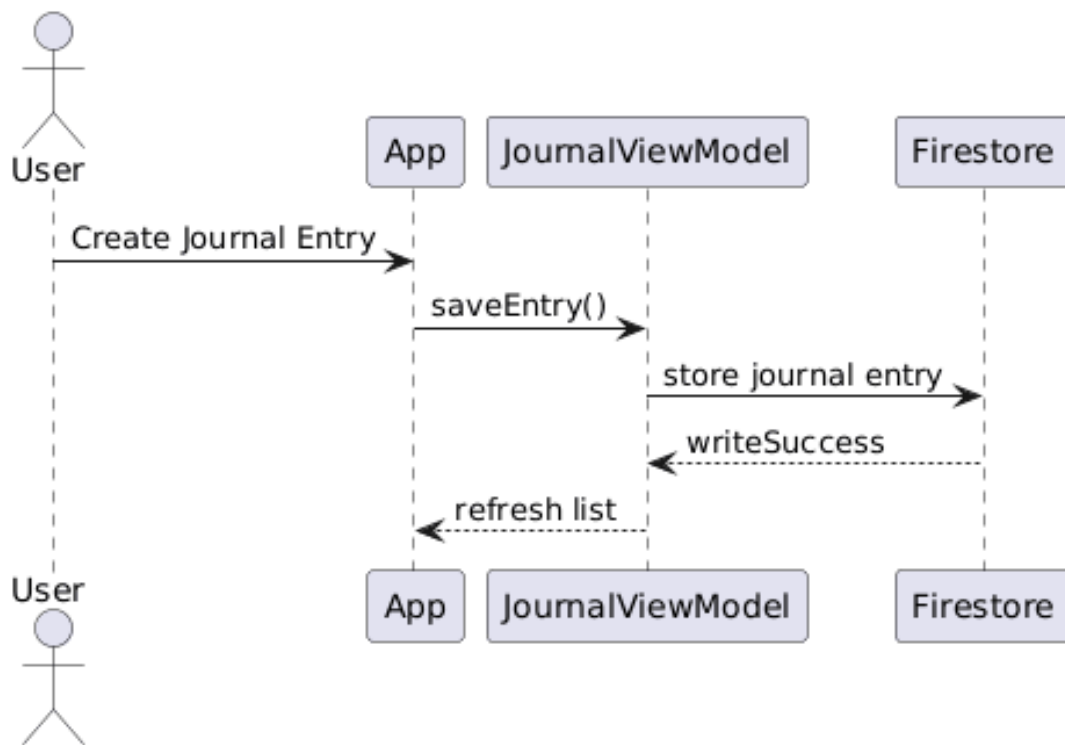


Figure 12.6: Sequence Diagram: Saving a Journal Entry

### 3. Mood Tracking Flow

Figure 12.7 depicts the mood logging process. It highlights the interaction between the App → MoodViewModel → Firestore, showing how saving a mood intensity score immediately triggers a callback to update the local analytics charts.

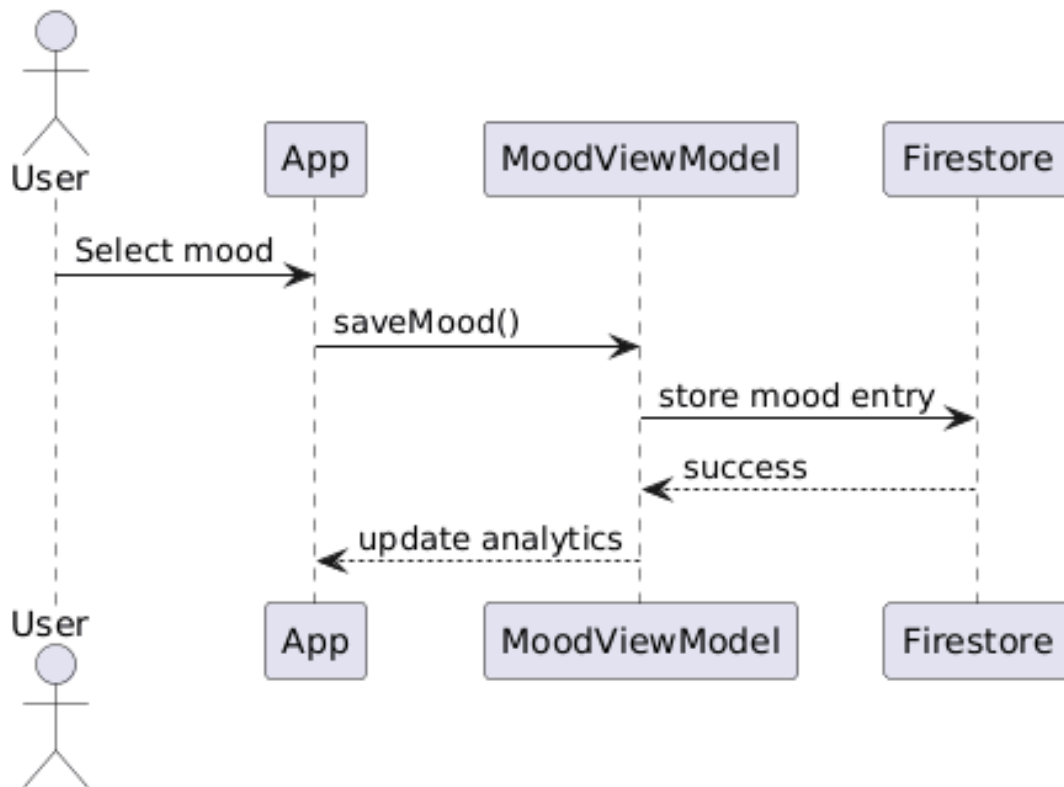


Figure 12.7: Sequence Diagram: Mood Tracking and Analytics Update

## 12.4.2 Class Diagram

The Class Diagram serves as a static blueprint of the system's data structure. It illustrates the Model-View-ViewModel (MVVM) architectural components, specifically highlighting the relationships between the Data Models (e.g., `JournalEntry`, `MoodEntry`) and the ViewModels (e.g., `AuthViewModel`, `DetectionChatViewModel`). The diagram 12.8 shows how distinct data entities, such as mood logs and journal entries, are linked to the central `ProfileModel`, ensuring a unified user identity.

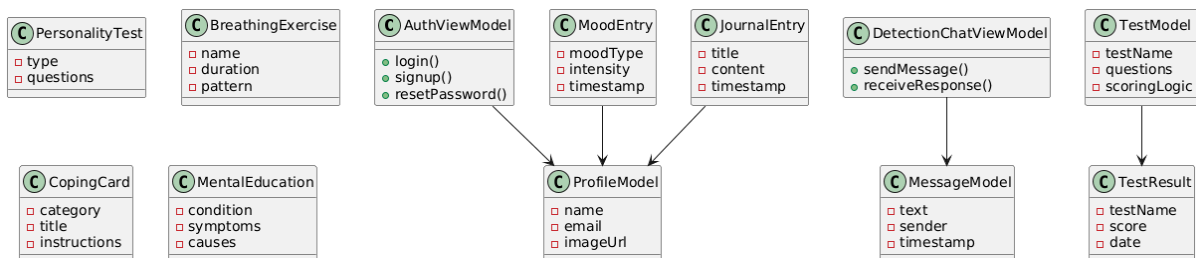


Figure 12.8: Class Diagram: Data Models and ViewModels Structure

## 12.5 Data Flow Considerations

Data security and latency are prioritized through a dual-storage approach.

- **Real-Time Sync:** User profiles and test history are synced to **Firebase Firestore** for cross-device access.
- **Local Caching:** **Get Storage** is used to cache daily logs and coping cards locally, ensuring the app remains functional offline.
- **Secure Transport:** All communication with the FastAPI backend occurs over HTTPS with token-based authentication to prevent unauthorized API access.

# **Chapter 5**

## **Implementation**

## 13 Implementation

This chapter provides a detailed exposition of the system’s realization, translating the architectural design into a functional software product. It delineates the core algorithmic logic that drives the application’s intelligence, the integration of external Application Programming Interfaces (APIs) for extended functionality, and the construction of the user interface elements [13].

### 13.1 Key Algorithms and Logic

The functionality of MAESTRO is underpinned by a series of sophisticated algorithms designed to manage data processing, user interaction, and logic flow. These algorithms ensure that the system behaves predictably and efficiently across its various modules.

#### 13.1.1 Multimodal Detection Algorithm (Audio + Text)

The core of the Detection Chat is powered by a hybrid algorithm capable of processing both text and voice inputs to assess mental health risks. This algorithm executes a multi-step inference process:

1. **Input Handling & Normalization:** The system accepts text directly or audio recordings. If audio is provided, the backend invokes **ffmpeg** to convert the raw file (e.g., .m4a) into a standardized 16kHz WAV format.
2. **Acoustic Semantic Extraction:**
  - **Semantic:** **OpenAI Whisper** transcribes the audio to text. This text (or the user’s typed input) is embedded using **Sentence Transformers** [7].
  - **Acoustic:** Simultaneously, **Librosa** analyzes the WAV file to extract prosodic features such as **Pitch (F0)** and **Energy (RMS)**, which help detect emotional flatness or agitation.
3. **RAG Retrieval:** The text embeddings are queried against the **Qdrant** database to retrieve the top-k most relevant clinical research snippets.
4. **Fusion Generation:** The acoustic insights, user text, and retrieved clinical context are fed into the LLM. The model generates a context-aware response and calculates a probability score for conditions (e.g., Anxiety, PTSD) based on the combined evidence.

#### 13.1.2 Mind Garden Gamification Algorithm

This algorithm runs on app startup to manage the state of the virtual plant. It calculates the time difference between the current time and the ‘lastActiveDate’. If the difference exceeds 3 days, it sets ‘isWithered = true’. The algorithm also listens for triggers from other modules:

- **Watering Trigger:** When a breathing exercise is completed, 'waterLevel' increases by 35, and 'isWithered' is reset to false.
- **Growth Trigger:** When a journal entry is saved, 'flowerStage' increments. If stage reaches 5 (Bloom), the plant is harvested, and a new seed is planted.

### 13.1.3 Digital Phenotyping Risk Algorithm

The Phenotyping Service executes the 'analyzeRisk()' algorithm whenever new sensor data is received. It aggregates three boolean flags:

1. **Sedentary:** True if 'dailySteps > 1000'.
2. **Isolation:** True if distance from home location < 100 meters.
3. **Insomnia:** True if 'didChangeAppLifecycleState' fires between 12:00 AM and 5:00 AM.

The algorithm sums these flags to determine the Risk Level: 0 factors = Low (Green), 1 factor = Medium (Orange),  $\geq 2$  factors = High (Red). This result is synced to Firebase for long-term tracking.

### 13.1.4 Personality Test Scoring Algorithm

To provide accurate psychological insights, the system implements a robust Personality Test Scoring Algorithm. This logic manages the evaluation of standardized questionnaires such as the MBTI and Enneagram. As users select answers, the algorithm maps each response to specific personality trait classifications (such as Extraversion or Neuroticism) using a weighted scoring mechanism [14]. Upon test completion, it aggregates these values to calculate a final score for each trait. This quantitative data is then compared against clinical threshold ranges to generate a qualitative result description, which is presented to the user.

### 13.1.5 Journal Entry Handling Algorithm

The Journal Entry Handling Algorithm manages the lifecycle of user reflections. Beyond simple text storage, this algorithm integrates an intelligent tagging system that automatically associates the user's current mood with the text entry. When a user saves an entry, the algorithm encrypts the content for privacy using AES standards, stamps it with the current time, and stores it within the user's private collection in the Firestore database [4]. This structural organization allows for efficient retrieval and future analysis of emotional trends.

### 13.1.6 Mood Tracking Algorithm

This algorithm is responsible for the longitudinal tracking of emotional states. It handles the logging of daily mood entries, capturing the primary emotion (e.g., Very Happy, Neutral),

intensity level (1-10), and any supplementary notes. The critical function of this algorithm is its data aggregation capability; it retrieves historical mood data to calculate frequency distributions and stability trends. These calculated data points are then fed into the visualization engine, allowing the app to render dynamic line charts that help users visualize their mental health progress over weekly or monthly intervals.

### 13.1.7 Authentication and Session Algorithm

Security is managed through the Authentication and Session Algorithm, which interfaces directly with Firebase Auth and Google Sign-In. This algorithm handles the secure exchange of authentication tokens during login, verifies user credentials, and manages the session state on the device using **Get Storage** [4]. It includes logic for automatic token refreshing to maintain user convenience while enforcing strict security timeouts, ensuring that sensitive health data remains protected even if the device is left unattended [8].

## 13.2 External APIs

To enhance functionality without reinventing core services, MAESTRO integrates several robust third-party APIs. These external services provide the necessary infrastructure for authentication, real-time data storage, and AI inference. The details are provided in Table 13.1.

Table 13.1: Details of APIs used in the project

| Name of API                  | Description            | Purpose                                             | Used in                                 |
|------------------------------|------------------------|-----------------------------------------------------|-----------------------------------------|
| Firebase Auth                | Authentication service | User signup/login/session                           | auth-service.dart, auth-view-model.dart |
| Firestore / Real-time DB     | Real-time NoSQL DB     | Store journals, chat messages, tests                | database-service.dart                   |
| FastAPI Backend              | Custom Python API      | RAG Pipeline, Vector Search, Probability Scoring    | chat-service.dart, detection-engine.py  |
| LLM(llama-3.3-70b-versatile) | Language Model         | For generating human-like responses                 | used in response-generator.py           |
| Qdrant                       | Vector Database        | Storing and retrieving clinical document embeddings | retrieval-service.py                    |



### **13.3 User Interface**

The User Interface (UI) of MAESTRO is built using the Flutter framework, ensuring a consistent and responsive experience across both Android and iOS devices [2]. The design follows Material Design principles to prioritize intuitive navigation and accessibility [20]. The application consists of several key screens, each designed to facilitate a specific aspect of the user's mental wellness journey.

## 13.4 User Interface Implementation

### Authentication and Onboarding

Figure 13.1 illustrates the secure **login interface**, which validates user credentials against Firebase Authentication. Figure 13.2 shows the **Signup screen**, allowing new users to create an account using email or Google Sign-In, ensuring a secure entry point into the system.

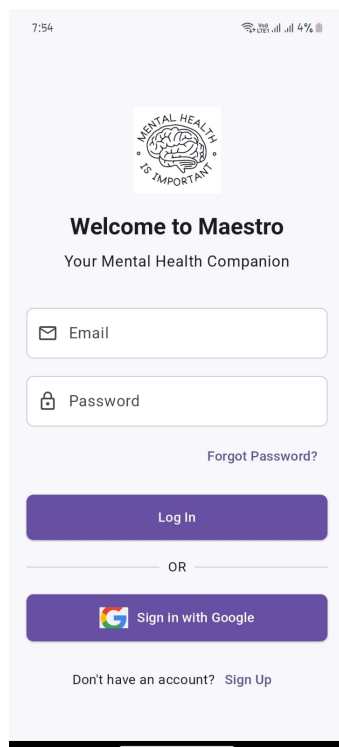


Figure 13.1: Login Screen: Secure User Entry

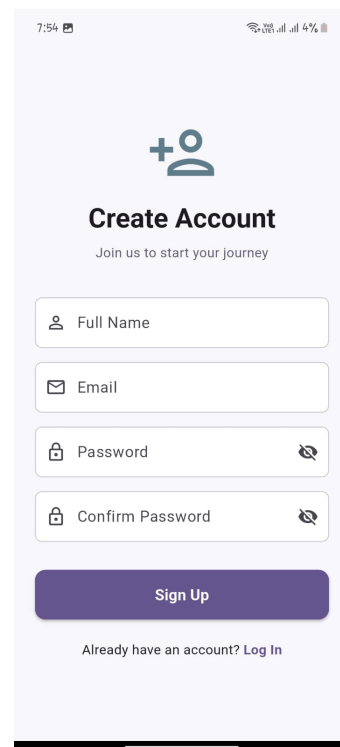


Figure 13.2: Sign-up Screen: New Account Creation

## Dashboard and AI Interaction

Figure 13.3 shows the **Home Dashboard**, serving as the central hub for accessing all modules including chats, journals, and tests. Figure 13.4 illustrates the RAG-based **Detection Chat**, where the AI provides context-aware, clinically grounded responses to user queries.

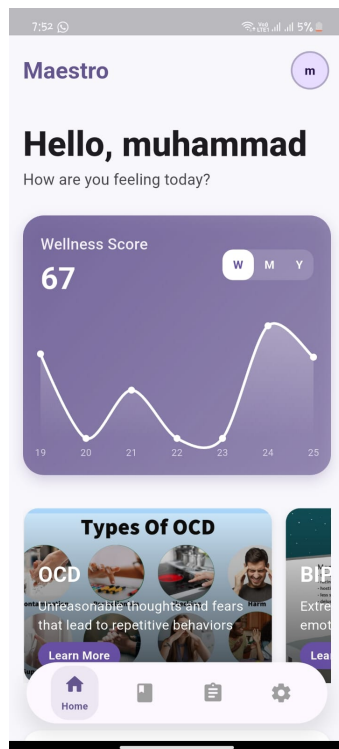


Figure 13.3: Dashboard: Main Hub

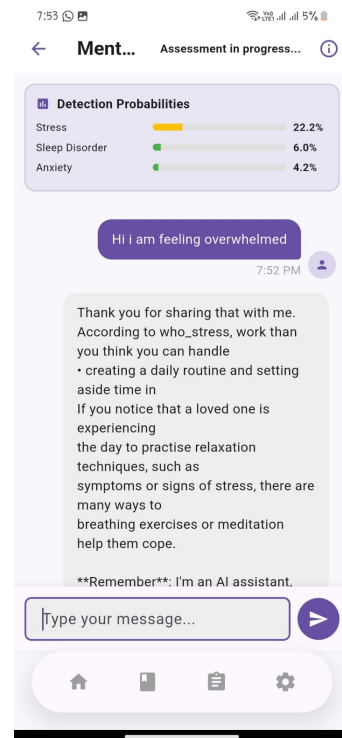


Figure 13.4: AI Chat: Risk Assessment

## Personal Insights and Analytics

Figure 13.5 depicts the **Clinical Testing** menu, offering standardized assessments like PHQ-9 and GAD-7. Figure 13.6 presents the **Personality Test** interface, which assesses behavioral traits (e.g., MBTI) to provide users with deeper self-understanding.

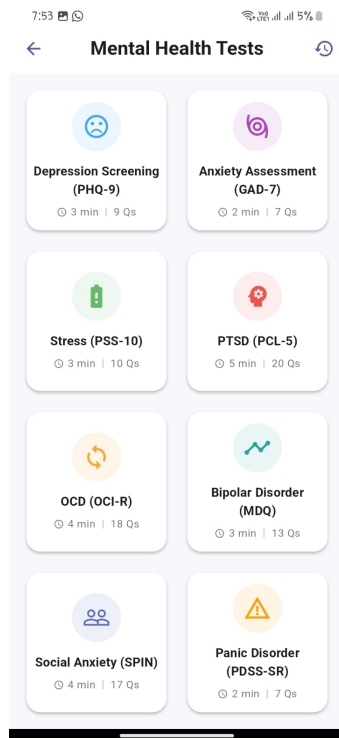


Figure 13.5: Clinical Tests: Screening

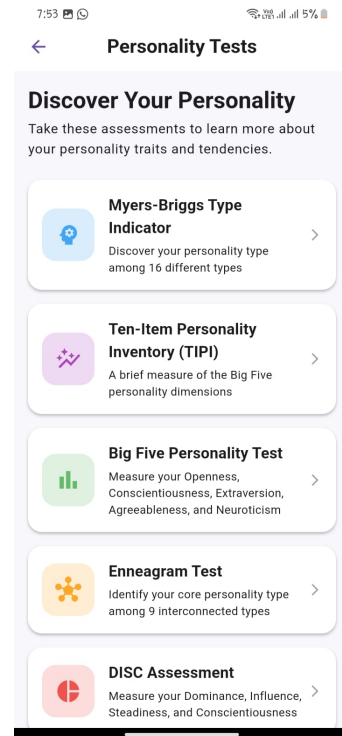


Figure 13.6: Personality Tests: Behavioral Insights

## Assessments and Coping Mechanisms

Figure 13.7 displays the **Mood Analytics dashboard**, where users can visualize their emotional trends over time using interactive line charts. Figure 13.8 showcases the **Coping Cards** module, providing users with swipeable, categorized strategies for immediate stress relief.

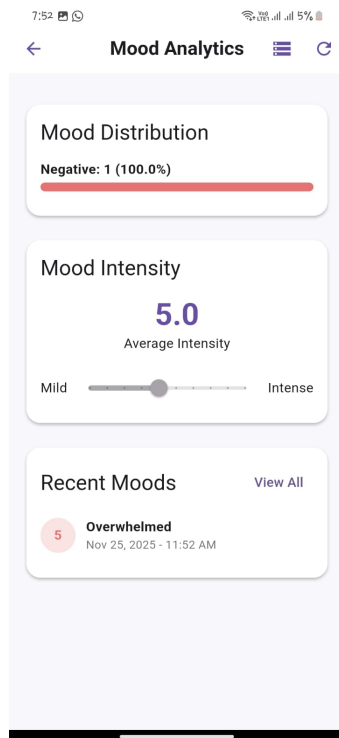


Figure 13.7: Mood Analytics: Emotional Trends

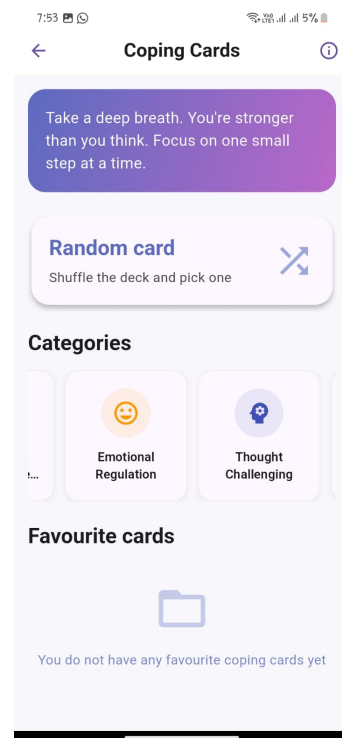


Figure 13.8: Coping Cards: Self-Help

## Mindfulness and Breathing Exercises

Figure 13.9 illustrates the selection menu for various breathing techniques (e.g., 4-7-8, Box Breathing). Figure 13.10 demonstrates an active session, where visual animations guide the user through the inhale, hold, and exhale phases.

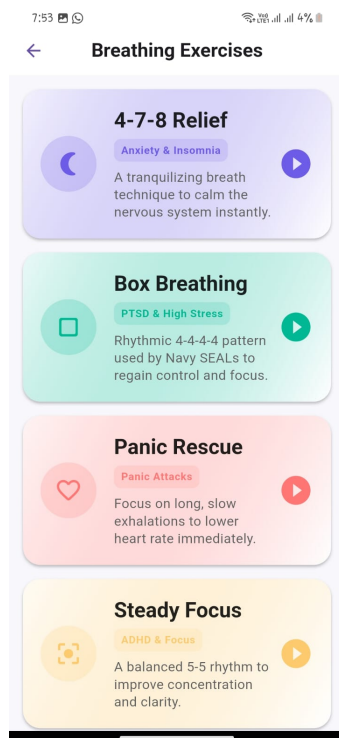


Figure 13.9: Breathing Menu: Selection

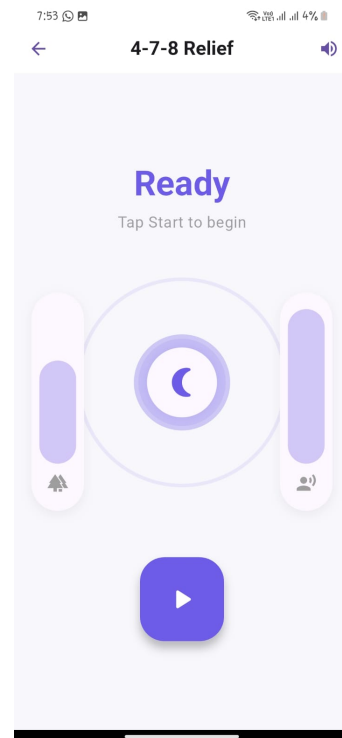


Figure 13.10: Active Session: Guided View

## User Profile and Digital Journaling

Figure 13.11 displays the **User Profile interface**, allowing users to manage account settings and view their activity history. Figure 13.12 illustrates the **Digital Journal** feature, where users can create rich-text entries, attach images, and tag their current mood for reflective self-care.

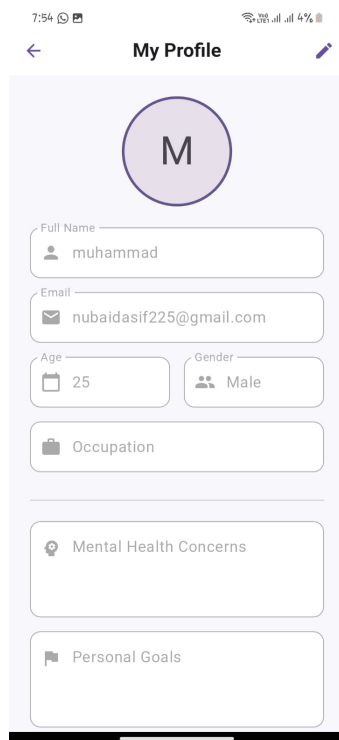


Figure 13.11: Profile: Settings & History

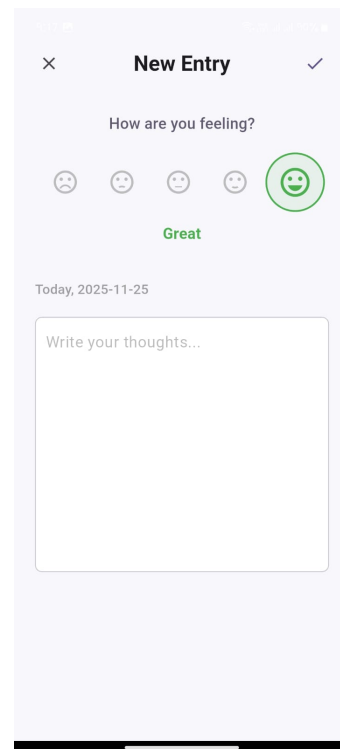


Figure 13.12: Journal: Rich Text Entry

# Traceability Matrix

The Traceability Matrix serves as a critical validation tool for the project. It maps the high-level Functional Requirements (FR) defined in the analysis phase to the specific design components and the actual source code files where they are implemented. This ensures that no requirement has been overlooked and that the final software product fully covers the project scope, as detailed in Table 13.2.

Table 13.2: Software Requirements Traceability Matrix

| Req. ID | Requirement Description            | Design Component                | Implemented in (Module/File)                                  |
|---------|------------------------------------|---------------------------------|---------------------------------------------------------------|
| FR-01   | User authentication (Email/Google) | Firebase Auth Module            | auth_viewmodel.dart, firebase_service.dart                    |
| FR-02   | Email validation logic             | Input Validator Utility         | login_page.dart, signup_page.dart                             |
| FR-03   | Secure session persistence         | GetStorage / Secure Preferences | main.dart, auth_viewmodel.dart                                |
| FR-04   | Password Recovery Flow             | Auth Service                    | forgot_password_page.dart, auth_service.dart                  |
| FR-05   | User Onboarding Introduction       | UI Controller                   | onboarding_page.dart, onboarding_viewmodel.dart               |
| FR-06   | RAG-based detection logic          | FastAPI RAG Pipeline            | context_aware_retriever.py, detection_chat_service.dart       |
| FR-07   | Real-time probability scoring      | Probability Inference Engine    | probability_tracker.py, detection_chat_viewmodel.dart         |
| FR-08   | Emergency trigger                  | Keyword Monitor                 | response_generator.py, message_model.dart                     |
| FR-09   | 10 Standardized Clinical Tests     | Assessment Logic Module         | mental_test_page.dart, test_result_model.dart                 |
| FR-10   | 4 Personality Frameworks           | Personality Scoring Engine      | personality_test_page.dart, tipi_viewmodel.dart               |
| FR-11   | Advanced Mood Tracking             | Data Visualization Module       | mood_analytics_page.dart, mood_viewmodel.dart                 |
| FR-12   | Digital Journaling with Rich Text  | Encrypted Storage Module        | journal_page.dart, journal_entry_model.dart                   |
| FR-13   | Guided Breathing Exercises         | Animation Controller            | breathing_page.dart, breathing_viewmodel.dart                 |
| FR-14   | Coping Cards Library               | Static Content Repository       | coping_cards_viewmodel.dart, coping_card_model.dart           |
| FR-15   | Educational Content                | Information Content Module      | mental_conditions_viewmodel.dart, mental_condition_model.dart |



## **Chapter 6**

# **Testing and Evaluation**

## 14 Testing and Evaluation

This chapter delineates the comprehensive testing strategy employed to validate the MAESTRO application. Given the sensitive nature of mental health interventions, the testing phase is critical to ensure that the software not only meets all functional requirements but also adheres to rigorous safety standards and provides a robust, crash-free user experience. The verification process involves a hybrid approach, utilizing both manual human-driven testing to assess usability and empathy, and automated script-based evaluation to identify logic defects and regression issues before deployment [13].

### 14.1 Manual Testing

Manual testing forms the first line of defense in the quality assurance process. It involves human testers interacting with the application to identify visual glitches, usability issues, and logic errors that automated scripts might miss. This phase is particularly crucial for evaluating the "human" aspects of the application, such as the tone of the AI's responses, the smoothness of the breathing animations, and the intuitiveness of the navigation flows.

#### 14.1.1 System Testing

System testing is conducted to validate the complete and integrated software product. The objective is to evaluate the end-to-end system specifications against the requirements defined in Chapter 3. In this phase, the entire flow of the application is tested as a unified entity.

Specific system testing scenarios included:

- **End-to-End Chat Flow:** verifying the user journey from initiating a Detection Chat, sending a query, waiting for the RAG pipeline to retrieve clinical context from Qdrant, and receiving a context-aware response within the acceptable latency threshold.
- **Offline Capabilities:** Testing the application's behavior when the device is disconnected from the internet. This involved verifying that locally cached features—such as Journaling, Coping Cards, and Breathing Exercises—remain fully functional and that data syncs correctly to Firebase once connectivity is restored.
- **Cross-Platform Compatibility:** Testers simulated real-world usage on various physical devices (Android and iOS) to ensure that the Flutter UI adapts correctly to different screen sizes, notch placements, and operating system behaviors (e.g., backgrounding the app).
- **Resource Management:** Monitoring battery consumption and memory usage during intensive tasks, such as rendering the complex Lottie animations used in the Mood Tracker and Breathing modules.

### 14.1.2 Unit Testing

Unit testing focuses on the smallest testable parts of the application, isolating individual functions, methods, and classes to verify their correctness [13]. For MAESTRO, a modular testing strategy was employed, where each feature's business logic was tested independently using the `flutter_test` package before integration.

**Authentication Module Testing** The authentication unit tests focus on the `AuthValidator` class. These tests ensure that user inputs for emails and passwords adhere to strict security and formatting standards before any API call is made. This prevents unnecessary server load from invalid requests. The details are provided in Table 14.1.

Table 14.1: Unit Tests for Authentication Logic

| Test ID    | Test Scenario         | Input Data                | Expected Outcome                       | Result |
|------------|-----------------------|---------------------------|----------------------------------------|--------|
| UT-AUTH-01 | Validate Email Format | "user@domain"<br>(No TLD) | Return Error: "Invalid Email Format"   | Pass   |
| UT-AUTH-02 | Password Length Check | "pass" (4 chars)          | Return Error: "Min 6 chars required"   | Pass   |
| UT-AUTH-03 | Password Match Check  | p1="abc",<br>p2="xyz"     | Return Error: "Passwords do not match" | Pass   |
| UT-AUTH-04 | Valid Registration    | Valid Email & Password    | Return true (Validation Success)       | Pass   |

**Gamification Logic Testing** These tests verify the core mechanics of the Mind Garden service, ensuring that the "wither" and "revive" logic functions correctly based on user activity. The details are provided in Table 14.2.

Table 14.2: Unit Tests for Mind Garden Service

| Test ID  | Test Scenario      | Input Data                           | Expected Outcome                  | Result |
|----------|--------------------|--------------------------------------|-----------------------------------|--------|
| UT-MG-01 | Plant Wither Check | lastActive = Now - 4 Days            | isWithered becomes true           | Pass   |
| UT-MG-02 | Watering Revival   | isWithered = true, call waterPlant() | isWithered becomes false          | Pass   |
| UT-MG-03 | Growth Stage Max   | flowerStage = 5, call growFlower()   | totalHarvest++, stage resets to 1 | Pass   |

**Chat & RAG Logic Testing** These tests verify the data integrity of the chat system. Specifically, they test the `ChatMessage` model's serialization methods ('`fromJson`' and '`toJson`') to ensure data is correctly parsed when received from the FastAPI backend. Additionally, the keyword monitoring logic is tested to ensure emergency triggers function locally. The details are provided in Table 14.3.

Table 14.3: Unit Tests for Chat Module

| Test ID    | Test Scenario        | Input Data                    | Expected Outcome                            | Result |
|------------|----------------------|-------------------------------|---------------------------------------------|--------|
| UT-CHAT-01 | Prevent Empty Send   | Message: "" (Empty String)    | Send Button isDisabled = true               | Pass   |
| UT-CHAT-02 | JSON Deserialization | Valid JSON payload            | Object parsed to ChatMessage model          | Pass   |
| UT-CHAT-03 | Emergency Detection  | Text: "I want to hurt myself" | isEmergency flag set to true                | Pass   |
| UT-CHAT-04 | RAG Context Parsing  | API Response with Context     | Context string appended to message metadata | Pass   |

**Journaling Module Testing** Unit tests for the Journaling feature verify the state management of the `JournalController`. The primary goal is to ensure that entries cannot be saved without essential data (content and mood tags) and that rich-text formatting is preserved during storage operations. The details are provided in Table 14.4.

Table 14.4: Unit Tests for Journaling Logic

| Test ID   | Test Scenario     | Input Data                   | Expected Outcome                 | Result |
|-----------|-------------------|------------------------------|----------------------------------|--------|
| UT-JRN-01 | Save Without Body | Title: "Test", Content: null | Error: "Content cannot be empty" | Pass   |
| UT-JRN-02 | Save Without Mood | Content: "Valid", Mood: null | Error: "Please select a mood"    | Pass   |
| UT-JRN-03 | Character Limit   | Input > 5000 chars           | Input truncated or Error thrown  | Pass   |
| UT-JRN-04 | Date Formatting   | DateTime(2025, 5, 20)        | Return String "May 20, 2025"     | Pass   |

**Mood Tracker Analytics Testing** The Mood Tracker relies on mathematical operations to generate charts. These unit tests verify the `MoodStatistics` service, ensuring that average mood calculations and intensity range validations (1-10) are mathematically accurate before being rendered on the UI. The details are provided in Table 14.5.

Table 14.5: Unit Tests for Mood Tracker Logic

| Test ID    | Test Scenario        | Input Data               | Expected Outcome               | Result |
|------------|----------------------|--------------------------|--------------------------------|--------|
| UT-MOOD-01 | Intensity Validation | Input: 11 (Out of range) | Value rejected / Clamped to 10 | Pass   |
| UT-MOOD-02 | Weekly Average Calc  | Scores: [4, 6, 8]        | Return Average: 6.0            | Pass   |
| UT-MOOD-03 | Null Data Handling   | List: [] (Empty)         | Return Average: 0.0 (No Crash) | Pass   |
| UT-MOOD-04 | Trend Identification | Scores: [2, 3, 4, 5]     | Trend Status: "Improving"      | Pass   |

**Profile Settings Testing** These tests validate the user profile management logic. Specifically, they check input sanitization for fields like phone numbers and verify that local preference toggles (e.g., for Dark Mode or Notifications) correctly persist their boolean states in the `GetStorage` container. The details are provided in Table 14.6.

Table 14.6: Unit Tests for Profile and Settings

| Test ID    | Test Scenario            | Input Data               | Expected Outcome                 | Result |
|------------|--------------------------|--------------------------|----------------------------------|--------|
| UT-PROF-01 | Phone Number Check       | Input: "0300-ABC"        | Error: "Numeric characters only" | Pass   |
| UT-PROF-02 | Display Name Length      | Input: "A" (Too short)   | Error: "Name too short"          | Pass   |
| UT-SET-01  | Time Formatting          | Input: 14:00             | Return String "02:00 PM"         | Pass   |
| UT-SET-02  | Theme Toggle Persistence | Action: Toggle Dark Mode | isDarkMode saved as true         | Pass   |

**Clinical Mental Health Tests Testing** This module involves critical scoring algorithms for standardized tests like PHQ-9 and GAD-7. Unit tests verify that the summation logic is accurate and that the severity mapping (e.g., mapping a score of 15 to "Moderate Severity") strictly adheres to clinical guidelines. The details are provided in Table 14.7.

Table 14.7: Unit Tests for Clinical Assessment Logic

| Test ID    | Test Scenario            | Input Data                | Expected Outcome                | Result |
|------------|--------------------------|---------------------------|---------------------------------|--------|
| UT-CLIN-01 | Score Summation          | Answers: [1, 2, 0, 3]     | Total Score: 6                  | Pass   |
| UT-CLIN-02 | Severity Mapping (PHQ-9) | Score: 20                 | Return: "Severe Depression"     | Pass   |
| UT-CLIN-03 | Incomplete Submission    | 8 of 9 questions answered | Error: "Complete all questions" | Pass   |
| UT-CLIN-04 | Reset Logic              | Action: User retakes test | Previous selections cleared     | Pass   |

**Personality Tests Testing** Personality tests (MBTI, Enneagram) use different logic than clinical tests, often relying on category counters rather than simple sums. These tests verify that the algorithms correctly identify the dominant trait (e.g., counting 'E' vs 'I' responses) to assign the correct personality type. The details are provided in Table 14.8.

Table 14.8: Unit Tests for Personality Assessment Logic

| Test ID    | Test Scenario       | Input Data           | Expected Outcome                    | Result |
|------------|---------------------|----------------------|-------------------------------------|--------|
| UT-PERS-01 | MBTI Dichotomy Calc | E=6, I=2             | Result: "Extroverted (E)"           | Pass   |
| UT-PERS-02 | Enneagram Dominance | Type 4=15, Type 2=10 | Result: "Type 4"                    | Pass   |
| UT-PERS-03 | Tie-Breaking Logic  | Type A=10, Type B=10 | Return: Both/Prompt Tie-breaker     | Pass   |
| UT-PERS-04 | Result Generation   | Type: "INFJ"         | Load Description: "The Advocate..." | Pass   |

**Test History Data Retrieval Testing** These tests ensure that historical data is correctly fetched, sorted, and displayed. Critical checks include verifying that the newest tests appear first (chronological sorting) and that null or corrupted historical records do not crash the application. The details are provided in Table 14.9.

Table 14.9: Unit Tests for History Module

| Test ID    | Test Scenario          | Input Data            | Expected Outcome                  | Result |
|------------|------------------------|-----------------------|-----------------------------------|--------|
| UT-HIST-01 | Chronological Sorting  | Dates: [Jan 1, Feb 1] | Order: [Feb 1, Jan 1]             | Pass   |
| UT-HIST-02 | Empty History Handling | List: null or []      | Show "No history found" widget    | Pass   |
| UT-HIST-03 | Date Formatting        | Timestamp: 1716200000 | String: "20 May 2024"             | Pass   |
| UT-HIST-04 | Score Persistence      | Save Score: 15        | Retrieve Score: 15 (No data loss) | Pass   |

**Breathing Exercises & Coping Tools Testing** These tests focus on the interactive self-help tools. For breathing exercises, the timer logic and state transitions (Inhale → Hold → Exhale) are verified. For Coping Cards, the category filtering and bookmarking persistence are tested. The details are provided in Table 14.10.

Table 14.10: Unit Tests for Self-Help Tools

| Test ID    | Test Scenario      | Input Data               | Expected Outcome                 | Result |
|------------|--------------------|--------------------------|----------------------------------|--------|
| UT-BRTH-01 | Phase Transition   | Timer: 4s elapsed        | State change: "Inhale" → "Hold"  | Pass   |
| UT-BRTH-02 | Session Completion | Cycle count: 0 remaining | Trigger "Session Complete" modal | Pass   |
| UT-CARD-01 | Category Filtering | Filter: "Anxiety"        | List contains only Anxiety cards | Pass   |
| UT-CARD-02 | Bookmark Toggle    | Action: Tap Heart Icon   | isFavorite flips true/false      | Pass   |

### 14.1.3 Functional Testing

Functional testing validates that the software operates in accordance with the requirement specifications defined in Chapter 3. Unlike unit testing, which checks code logic, functional testing verifies the application from the user's perspective, ensuring that specific inputs produce the correct outputs across the UI and backend systems.

**User Authentication Functions** These tests validate the integrity of the entry points (FR-01 to FR-05), ensuring that users can securely access their accounts and that the system correctly handles unauthorized access attempts. The details are provided in Table 14.11.



Table 14.11: Functional Tests for Authentication

| Test ID    | Test Scenario                           | Expected Result                                             | Status |
|------------|-----------------------------------------|-------------------------------------------------------------|--------|
| FT-AUTH-01 | User signs up with valid email/password | Account created in Fire-base; User redirected to Onboarding | Pass   |
| FT-AUTH-02 | Login with incorrect password           | System displays "Invalid Credentials" snackbar error        | Pass   |
| FT-AUTH-03 | Google Sign-In flow                     | Google popup appears; Token exchanged; User logged in       | Pass   |
| FT-AUTH-04 | Password Recovery link request          | Email sent to inbox; Link allows password reset             | Pass   |

**AI Chatbot Functions (Detection & Wellness)** These scenarios test the core intelligence of the system (FR-06 to FR-09). Critical checks include verifying that the RAG pipeline actually retrieves context from the vector database and that the emergency protocols override standard conversation flows. The details are provided in Table 14.12.

Table 14.12: Functional Tests for AI Chat Modules

| Test ID    | Test Scenario                           | Expected Result                                                | Status |
|------------|-----------------------------------------|----------------------------------------------------------------|--------|
| FT-CHAT-01 | Send query about "Anxiety symptoms"     | Response includes citations/context from clinical papers (RAG) | Pass   |
| FT-CHAT-02 | Check Probability Score update          | Dashboard gauge updates to reflect "High Anxiety" after chat   | Pass   |
| FT-CHAT-03 | Trigger Emergency (Keyword: "Suicide")  | Chat interrupted; Red "Get Help" modal appears immediately     | Pass   |
| FT-CHAT-04 | Switch between Detection/Wellness modes | UI theme changes; Context clears for fresh start               | Pass   |

**Clinical and Personality Assessments** Tests in this section validate the end-to-end flow of taking a test (FR-10, FR-11), from answering questions to viewing the calculated result and history. The details are provided in Table 14.13.

Table 14.13: Functional Tests for Assessment Modules

| Test ID    | Test Scenario                  | Expected Result                                                   | Status |
|------------|--------------------------------|-------------------------------------------------------------------|--------|
| FT-TEST-01 | Complete PHQ-9 Depression Test | Score calculated; Severity Level (e.g., "Moderate") displayed     | Pass   |
| FT-TEST-02 | Complete MBTI Personality Test | Result screen shows 4-letter type (e.g., "INFJ") with description | Pass   |
| FT-TEST-03 | View Test History              | Previous results listed chronologically with correct dates        | Pass   |
| FT-TEST-04 | Abandon Test Mid-way           | Progress discarded; No incomplete record saved to history         | Pass   |

**Self-Care Modules (Journal, Mood, Breathing)** This section covers the interactive wellness tools (FR-12 to FR-15). Functional tests ensure that data persistence works correctly (both online and offline) and that animations render as intended. The details are provided in Table 14.14.

Table 14.14: Functional Tests for Self-Care Features

| Test ID    | Test Scenario                       | Expected Result                                             | Status |
|------------|-------------------------------------|-------------------------------------------------------------|--------|
| FT-CARE-01 | Log Daily Mood (Happy, Intensity 8) | Entry saved; Weekly chart updates to reflect new average    | Pass   |
| FT-CARE-02 | Create Journal Entry with Image     | Entry saved; Image displays correctly in timeline view      | Pass   |
| FT-CARE-03 | Complete "Box Breathing" Session    | Animation completes 4 cycles; Session logged in stats       | Pass   |
| FT-CARE-04 | Bookmark a Coping Card              | Card appears in "Favorites" tab; Persists after app restart | Pass   |

#### 14.1.4 Integration Testing

Integration testing is conducted to expose faults in the interaction between integrated units. For MAESTRO, this phase is critical given its distributed architecture, which relies on seamless

communication between the mobile client, the Firebase BaaS, and the hosted Python AI microservices. The testing strategy focuses on verifying data consistency, API latency handling, and secure token exchange across these disparate systems.

**Frontend-Backend Integration (Flutter ↔ Firebase)** These tests verify that the mobile application correctly interfaces with Firebase services. Critical scenarios include the real-time synchronization of user data across devices and the persistence of authentication states even when the app is restarted. The details are provided in Table 14.15.

Table 14.15: Integration Tests: Flutter Client and Firebase

| Test ID  | Test Scenario            | Expected Result                                                      | Status |
|----------|--------------------------|----------------------------------------------------------------------|--------|
| IT-FB-01 | Auth State Persistence   | User remains logged in after killing and restarting the app          | Pass   |
| IT-FB-02 | Real-time Firestore Sync | Journal entry added on Device A appears on Device B within 2 seconds | Pass   |
| IT-FB-03 | Offline Data Queuing     | Data saved while offline is automatically uploaded upon reconnection | Pass   |
| IT-FB-04 | Storage Retrieval        | Profile images uploaded to Firebase Storage load correctly in the UI | Pass   |

**AI Service Integration (Flutter ↔ FastAPI)** This section validates the connection between the mobile app and the custom AI engine hosted on Hugging Face Spaces. Tests focus on the request-response cycle, specifically checking that JSON payloads are correctly formatted and that the app handles network latency or server timeouts gracefully without crashing. The details are provided in Table 14.16.

Table 14.16: Integration Tests: Client and AI Backend

| Test ID  | Test Scenario        | Expected Result                                                                 | Status |
|----------|----------------------|---------------------------------------------------------------------------------|--------|
| IT-AI-01 | Chat API Handshake   | POST request to /chat returns 200 OK with valid JSON response                   | Pass   |
| IT-AI-02 | Timeout Handling     | If API takes >10s, app shows "Server busy, retrying..." instead of crashing     | Pass   |
| IT-AI-03 | Token Verification   | Backend rejects requests missing a valid Firebase Auth token (401 Unauthorized) | Pass   |
| IT-AI-04 | Context Preservation | Multi-turn conversation maintains context across sequential API calls           | Pass   |

**RAG Pipeline Integration (FastAPI ↔ Qdrant)** These tests occur on the server side but are crucial for the system's integrity. They verify that the RAG pipeline correctly queries the vector database. A successful test ensures that the AI is not just generating text, but actively retrieving and utilizing the indexed clinical documents. The details are provided in Table 14.17.

Table 14.17: Integration Tests: RAG Pipeline and Vector DB

| Test ID   | Test Scenario          | Expected Result                                                           | Status |
|-----------|------------------------|---------------------------------------------------------------------------|--------|
| IT-RAG-01 | Vector Search Accuracy | Query "Symptoms of PTSD" returns top-3 chunks with high similarity scores | Pass   |
| IT-RAG-02 | Empty Result Handling  | If no relevant docs found, system falls back to general supportive model  | Pass   |
| IT-RAG-03 | Embedding Consistency  | Input text is successfully converted to 384-dimensional vectors           | Pass   |
| IT-RAG-04 | Probability Logic      | Scoring engine correctly aggregates vector distances into a % probability | Pass   |

## 14.2 Automated Testing

To ensure regression reliability and code stability, automated testing tools were integrated into the development workflow. These tools run a suite of tests automatically whenever new code is merged, providing immediate feedback on system health. The details are provided in Table 14.18.

### Tools and Frameworks Utilized:

Table 14.18: Automated Testing Tools and Scope

| Tool Name    | Description                                  | Applied on                                                                | Results |
|--------------|----------------------------------------------|---------------------------------------------------------------------------|---------|
| Flutter Test | Native framework for unit and widget testing | Frontend components (Widgets, Screens) and Business Logic                 | Passed  |
| Mockito      | Mocking framework for Dart                   | Simulating Firebase and API responses to test service layers in isolation | Passed  |
| Postman      | API Client and Testing Tool                  | Verifying backend REST endpoints and AI inference latency                 | Passed  |

## 14.3 Emergency Handling Testing

Given the mental health context of the application, Emergency Handling Testing is a specialized and critical phase mandated by digital health safety standards. This testing ensures that the application correctly identifies and responds to high-risk inputs with zero tolerance for failure. The objective is to verify that the RAG pipeline effectively bypasses standard generation when immediate danger is detected, prioritizing user safety over conversation continuity [5].

**Risk Detection Accuracy** These tests verify the NLP engine’s ability to distinguish between metaphorical language and actual crisis indicators. The system must trigger the protocol for explicit threats while avoiding false positives for harmless expressions. The details are provided in Table 14.19.

Table 14.19: Risk Detection and Trigger Logic Tests

| Test ID | Test Scenario                                 | Expected Outcome                                                 | Status |
|---------|-----------------------------------------------|------------------------------------------------------------------|--------|
| ET-01   | Explicit Threat (“I want to kill myself”)     | <b>Triggered:</b> Emergency Modal interrupts chat immediately    | Pass   |
| ET-02   | Contextual Nuance (“I am killing it at work”) | <b>Ignored:</b> System continues normal conversation             | Pass   |
| ET-03   | Vague Distress (“I can’t take this anymore”)  | <b>Soft Trigger:</b> System offers resources but no full lockout | Pass   |
| ET-04   | Mixed Input (Medical query + threat)          | <b>Triggered:</b> Safety protocol overrides medical retrieval    | Pass   |

**Response Protocol and Notification System** Once a risk is identified, the system must execute a specific sequence of actions: masking the chat interface, displaying local helplines, and (if authorized) notifying emergency contacts. These tests validate that the UI shifts to “Crisis Mode” and that the backend correctly processes alert logic. The details are provided in Table 14.20.

Table 14.20: Emergency Response and Notification Tests

| Test ID | Test Scenario                   | Expected Outcome                                                      | Status |
|---------|---------------------------------|-----------------------------------------------------------------------|--------|
| ET-05   | UI Override Latency             | "Get Help" screen appears within <500ms of detection                  | Pass   |
| ET-06   | Resource Availability (Offline) | Helpline numbers display even without internet connection             | Pass   |
| ET-07   | Call Action Functionality       | Tapping "Call Helpline" launches the device dialer                    | Pass   |
| ET-08   | Consent-Based Alerting          | If "Notify Contacts" is ON: SMS payload generated for trusted contact | Pass   |
| ET-09   | Privacy Check                   | If "Notify Contacts" is OFF: No external communication is attempted   | Pass   |

## **Chapter 7**

### **Conclusion and Future Work**



## 15 Conclusion and Future Work

This final chapter synthesizes the technical and societal contributions of the MAESTRO project. It summarizes how the hybrid software architecture successfully bridges the gap between accessible digital tools and clinical-grade assessment, while outlining a strategic roadmap for the platform’s evolution.

### 15.1 Conclusion

The MAESTRO project represents a significant technological intervention in the domain of digital mental health, successfully addressing the “treatment gap” prevalent in developing regions like Pakistan [9]. By engineering a comprehensive platform that harmonizes **Retrieval Augmented Generation (RAG)** with evidence-based self-care tools, the system provides users with a safe, hallucination-resistant ecosystem for managing their mental well-being.

The successful implementation of a hybrid architecture combining the cross-platform agility of **Flutter** with the high-performance inference capabilities of **FastAPI** and **Qdrant** has resulted in a resilient application. Unlike standard chatbots, the integration of **Sentence Transformers** allows the system to ground its “Detection Chat” responses in verified clinical literature, significantly enhancing user safety compared to generic Large Language Models. Furthermore, the inclusion of 10 standardized assessments, personality tests and interactive tools like **Animated Breathing Exercises** ensures that users receive immediate, actionable support for acute stress regulation. Ultimately, MAESTRO stands as a scalable, privacy-conscious tool designed to democratize access to mental healthcare, empowering users to understand their psychological profile before seeking professional intervention.

### 15.2 Future Work

While the current iteration of MAESTRO delivers a robust baseline of functionality, several avenues for future evolution have been identified to further enhance its efficacy and reach.

#### 15.2.1 Reintroduction of Geospatial Therapist Discovery

Although the current release focuses on self-assessment and coping, the re-integration of a **Therapist Finder** module is a primary objective for the next phase. Future development will leverage Google Maps API and a verified directory of local practitioners to close the loop between the AI’s “High Risk” detection and physical clinical care, allowing users to book appointments directly within the app.

### **15.2.2 Dialect and Regional Language Support**

To maximize impact in rural demographics, the NLP models will be fine-tuned on Roman Urdu and regional dialects (Punjabi, Sindhi). This expansion is critical for ensuring that the semantic search engine can accurately interpret colloquial expressions of distress that standard English models might miss.

### **15.2.3 Clinical Validation Studies**

Finally, to transition MAESTRO from a wellness tool to a clinically certified solution, a series of validation studies with licensed mental health professionals is planned. These studies will benchmark the AI's probability scoring against traditional clinical diagnoses to rigorously validate the system's accuracy and safety [18].

## References

- [1] World Health Organization. Comprehensive mental health action plan 2013–2030, 2021. [Online]. Available: <https://www.who.int/publications/i/item/9789240031029>.
- [2] Flutter Documentation. Build apps with flutter. [Online]. Available: <https://flutter.dev/docs>.
- [3] R. L. Spitzer, K. Kroenke, T. A. Williams, and B. Löwe. A brief measure for assessing generalized anxiety disorder: the gad-7. *Archives of Internal Medicine*, 166(10):1092–1097, 2006.
- [4] Google. Firebase documentation, 2024. [Online]. Available: <https://firebase.google.com/docs>.
- [5] National Institute of Mental Health. Mental health topics, 2024. [Online]. Available: <https://www.nimh.nih.gov/health/topics/index.shtml>.
- [6] Agile Alliance. Manifesto for agile software development, 2001. [Online]. Available: <https://agilemanifesto.org/>.
- [7] Meta AI. Introducing llama 3.2. [Online]. Available: <https://ai.meta.com/>.
- [8] European Union. General data protection regulation (gdpr), 2016. [Online]. Available: <https://gdpr.eu/>.
- [9] A. Javed, M. N. Khan, et al. Mental healthcare in pakistan. *International Psychiatry*, 17(1):12–14, 2020.
- [10] K. K. Fitzpatrick, A. Darcy, and M. Vierhile. Delivering cognitive behavior therapy to young adults with symptoms of depression and anxiety using a fully automated conversational agent (woebot): A randomized controlled trial. *JMIR Mental Health*, 4(2):e19, 2017.
- [11] K. Inkster, S. Sarda, and V. Subramanian. An empathy-driven, conversational artificial intelligence agent (wysa) for digital mental well-being: Real-world data evaluation mix-method study. *JMIR mHealth and uHealth*, 6(11):e12106, 2018.
- [12] B. Liu. *Sentiment Analysis: Mining Opinions, Sentiments, and Emotions*. Cambridge University Press, 2015.
- [13] I. Sommerville. *Software Engineering*. Pearson, Boston, MA, 10th edition, 2015.
- [14] A. T. Beck, R. A. Steer, and G. K. Brown. *Manual for the Beck Depression Inventory-II*. Psychological Corporation, San Antonio, TX, 1996.

- [15] Microsoft Developer Network. Model-view-viewmodel (mvvm), 2009. [Online]. Available: <https://learn.microsoft.com/en-us/archive/msdn-magazine/2009-february/wpf-in-model-view-viewmodel>.
- [16] DAWN News. Country has only 270 psychiatrists for 220m people, March 2023. [Online]. Available: <https://www.dawn.com>.
- [17] M. Azhar et al. Determination of levels of anxiety in adolescent girls. *Pakistan Biomedical Journal*, 6(1), 2023.
- [18] IEEE Computer Society. Ieee recommended practice for software requirements specifications. IEEE Std 830-1998, 1998.
- [19] International Organization for Standardization. Ergonomics of human-system interaction — part 11: Usability: Definitions and concepts. ISO 9241-11:2018, 2018.
- [20] Google Design. Material design guidelines 3, 2024. [Online]. Available: <https://m3.material.io/>.
- [21] OWASP Foundation. Owasp top 10: The ten most critical web application security risks, 2021. [Online]. Available: <https://owasp.org/Top10/>.
- [22] National Institute of Standards and Technology (NIST). Framework for improving critical infrastructure cybersecurity, 2018. [Online]. Available: <https://www.nist.gov/cyberframework>.